

Ruben Apablaza Muñoz

PROYECTO
LABS
CIBERSEGURIDAD

LAB4
Hardening Avanzado
de Linux

UFW + Fail2Ban + CrowdSec + RH Hunter

Implementación en Ubuntu Server

– 0Zone –

⚠ **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD** ⚠

Este documento tiene fines **estrictamente educativos y de formación profesional** en el área de la ciberseguridad. El autor no se hace responsable por el mal uso de la información, scripts o comandos aquí descritos.

- **Entornos de prueba**

Se recomienda ejecutar este laboratorio exclusivamente en entornos controlados (Máquinas Virtuales o Contenedores). Nunca ejecute scripts o comandos desconocidos en servidores de producción sin antes probarlos y entender su funcionamiento.

- **Responsabilidad del usuario**

El uso de herramientas de nivel administrativo (`sudo`) conlleva riesgos. El lector es el único responsable de cualquier pérdida de datos o interrupción del servicio que pudiera derivarse de la aplicación de este manual.

- **Ética profesional**

Las técnicas de análisis aquí descritas deben ser utilizadas siempre bajo un marco legal y con la autorización debida.

Contenido

Introducción	5
Requisito de operación	7
1. Verificar servicios instalados.	9
2. UFW. Reiniciar configuración del firewall.	10
3. UFW. Aplicar política segura (principio de mínimo privilegio).....	11
4. UFW. Abrir servicios necesarios.....	12
5. UFW. Limitar ataques SSH.....	13
6. UFW. Activar firewall.	14
6.1 Limitar ICMP.	15
5.1.1 Añadir excepciones a la regla.....	17
7. Activar logs detallados.....	19
7.1. Expandir espacio de disco (lsblk).....	20
7.2 Configurar logs detallados.	22
8. Fail2Ban Configuración.	23
9. Fail2ban. Ver IPs bloqueadas.....	25
9.1 Prueba de fuerza bruta con Hydra	26
9.2 Desbloquear una IP bloqueada.....	27
10. Fail2ban. Bloqueo agresivo contra fuerza bruta.	28
11. Fail2ban. Detectar portscan.....	30
3. Agresividad y severidad	31
11.1 Agregar reglas para detección de escaneo de puertos.....	32
12. Fail2ban. Bloquear IP reincidentes (recidive)	33
12.1 IMPORTANTE. Ignorar direcciones IP.....	34
13. Fail2ban. Detectar IP que escanean muchos puertos	35
14. UFW. Protección contra SYN flood.....	37

15. CrowdSec (IDS/IPS colaborativo).....	39
15.1 IMPORTANTE. Ignorar direcciones IP (Whitelist).	44
15.2 Detectar escaneos comunes de Nmap y otros	46
15.3 Desbloquear IPs bloqueadas.	48
15.4 Activar el Dashboard visual.....	49
16. Logs: Del análisis manual al automatizado.....	54
16.1. Método tradicional.....	54
16.2. Método moderno.	54
16.3. Tabla comparativa entre métodos.	55
17. Monitoreo: Comandos "Old School" vs. "Pro".....	56
17.1. Monitoreo de red (Método tradicional)	56
17.2. Detección de ataques (Método tradicional)	56
17.2.1 Detección de ataques (Método moderno)	57
18. Cambiar puerto SSH.....	58
19. Desactivar root.	58
20. Usar autenticación por llaves.....	59
20.1. Generar las llaves en nuestra máquina local (Windows)	59
20.2. Copiar la llave al servidor Ubuntu	60
20.3. Permisos en el servidor.....	62
20.4. Prueba de oro	63
20.5 Deshabilitar contraseñas.....	64
20.6 Si no resulta	65
21. Instalar detector de rootkits (RKHunter).	66
21.1 Instalación.....	66
21.2. Configuración de la "Línea Base" (Baseline)	69
21.2.1. Actualizamos la base de datos interna de rkhunter	69
21.2.2 Captura las propiedades de nuestros archivos actuales (el "estado sano")	73
21.2.3 "El gran escaneo"	74
22. Monitoreo centralizado (script de Auditoría)	78
23. Port Knocking ¿Porque no está en esta guía?	91

23. SEGURIDAD V/S FUNCIONALIDAD	93
24. Comandos (Actualización Lab 3)	99
Fase 1: HIGIENE BASE Y AUTOMATIZACIÓN	99
Fase 2: GESTIÓN DE IDENTIDADES Y PRIVILEGIOS	99
Fase 3: HARDENING DE SSH	100
Fase 4: FIREWALL (UFW).....	100
Fase 5: DEFENSA ACTIVA (FAIL2BAN + CROWDSEC).....	101
Fase 6: INTEGRIDAD Y AUDITORÍA (LYNIS + RKHUNTER).....	101
Fase 7: MONITOREO Y FORENSE	102
25. Checklists	103
25.1. Checklist de mantención y monitoreo continuo	103
25.2. Checklist de sospechas de intrusión y respuesta a incidentes	104
25.3. Checklist de auditoría forense / post-incidente	105
26. Matriz de riesgos	106
27. Arquitectura del laboratorio.....	107
28. Conclusiones de seguridad	108

Introducción

Del hardening básico a la fortaleza activa

La seguridad de un servidor Linux no es un estado estático, sino una evolución constante. Mientras que el Laboratorio 3 sentó las bases críticas mediante la “*Higiene base*” (actualización de repositorios, eliminación de dependencias y parcheo automático con *unattended-upgrades*), el Laboratorio 4 representa la transición de un sistema “limpio” a lo que se conoce como “*Sistema hardenizado*” bajo el modelo de “*Defensa en Profundidad*”.

Esta evolución (Lab 4) no sustituye las medidas previas (Lab 3), sino que las dota de inteligencia y resiliencia, atacando los vectores de vulnerabilidad desde tres pilares complementarios:

1. Evolución de la identidad: De la contraseña a la criptografía

Si en la fase básica restringimos el acceso *root*, en esta fase consolidamos la eliminación de la superficie de ataque en SSH. Al migrar de una autenticación basada en contraseñas hacia la autenticación criptográfica de llave pública (ED25519), neutralizamos de raíz los ataques de fuerza bruta y diccionarios, garantizando que solo dispositivos autorizados pueden acceder.

2. Sinergia de defensa: Inteligencia local y global

La seguridad perimetral básica de UFW evoluciona hacia una gestión inteligente de amenazas. Aquí, las herramientas se complementan en un flujo de trabajo dinámico:

- **Fail2Ban** actúa como el “guardia local”, reaccionando a patrones de ataque detectados en los logs del servidor.
- **CrowdSec** eleva esta defensa al nivel de “*inteligencia colectiva*”, bloqueando preventivamente IPs maliciosas reportadas globalmente por otros servidores antes de que siquiera intenten tocar nuestra red.

3. Integridad y resiliencia estructural

Para que un sistema sea seguro, debe ser capaz de auditarse a sí mismo y resistir su propio crecimiento.

- **Auditoría interna:** Implementamos **RKHunter** y **Lynis** como capas de inspección profunda. Si un atacante lograra evadir el perímetro, estas herramientas detectarían alteraciones en binarios esenciales o la presencia de *rootkits*, garantizando la integridad del sistema.
- **Escalabilidad logística:** Mediante el uso de **LVM (Logical Volume Management)**, preparamos la infraestructura para soportar el High Logging de UFW. Esto asegura que el aumento en la generación de evidencias forenses no comprometa la estabilidad del almacenamiento ni provoque una denegación de servicio por disco lleno.

Conclusión del ecosistema: Al fusionar la higiene del Laboratorio 3 con las herramientas avanzadas del Laboratorio 4, el servidor deja de ser un objetivo pasivo. Se convierte en una infraestructura capaz de resistir ataques automatizados, detectar anomalías internas en tiempo real y colaborar con la seguridad global de la red.

Tenemos ahora una base de seguridad sumamente sólida:

la barrera (UFW), el guardia local (Fail2Ban), la inteligencia global (CrowdSec) y el auditor interno (RKHunter/Lynis).

Requisito de operación

Esta guía es la continuación estratégica de los laboratorios:

- **Lab 1: Instalación y preparación de Ubuntu Server.**
- **Lab 2: Análisis táctico de logs en sistemas Linux.**
- **Lab 3: Hardening Básico de Linux.**

En ciberseguridad, la integridad del entorno es fundamental. Para garantizar que las acciones que vamos a realizar hoy sean consistentes y que las configuraciones de red no interfieran con tus hallazgos, es **altamente recomendable** haber completado la fase previa.

Si aún no tienes tu entorno listo o quieres asegurarte de que tu máquina cumple con los estándares de este proyecto, puedes realizar ambos laboratorios aquí:

Haz *click* en la imagen para acceder al **Lab** correspondiente.



Laboratorio 4: Hardening AVANZADO de Linux: UFW + Fail2Ban + CrowdSec + RK Hunter

Objetivos:

- Reducir superficie de ataque
- Bloquear fuerza bruta
- Detectar escaneos
- Bloquear bots automatizados
- Integrar inteligencia de amenazas
- Generar logs para análisis de seguridad

UFW (Uncomplicated Firewall)

Es el **portero**. Nosotros le decimos qué puertas (puertos) están abiertas y cuáles cerradas. Si no está en la lista, no pasa.

Fail2Ban

Es el **guardia reactivo**. Vigila los intentos de inicio de sesión fallidos en nuestros archivos de registro. Si alguien se equivoca mucho, lo banea por un tiempo basándose en reglas locales.

CrowdSec

Es la **inteligencia colectiva**. Funciona como Fail2Ban (detecta comportamientos maliciosos), pero comparte esa información con una red global. Si una IP es bloqueada en otro país por atacar, CrowdSec la bloquea en nuestro sistema preventivamente.

RKHunter (Rootkit Hunter)

Es el **auditor interno**. Inspecciona las "entrañas" del servidor buscando programas ocultos (*rootkits*), puertas traseras y archivos sospechosos. Compara los archivos críticos del sistema contra una base de datos de firmas para asegurar que nada haya sido alterado por un intruso.

1. Verificar servicios instalados.

En el laboratorio anterior dejamos instalados y configurados *ufw* y *fail2ban*. Si no los tienes instalados puedes referirte a ese documento. Por lo tanto en este caso solo realizaremos la comprobación de que ambos servicios están OK.

Comprobar firewall y sistema de bloqueo.

```
sudo systemctl status ufw
sudo systemctl status fail2ban
```

Activarlos si no están activos.

```
sudo systemctl enable ufw
sudo systemctl enable fail2ban
```



```
ozone@ubuntuuser:~$ sudo systemctl status ufw
[sudo] password for ozone:
● ufw.service - Uncomplicated firewall
   Loaded: loaded (/usr/lib/systemd/system/ufw.service; enabled; preset: enabled)
   Active: active (exited) since Sat 2026-03-07 10:32:09 -03; 2min 13s ago
     Docs: man:ufw(8)
   Process: 620 ExecStart=/usr/lib/ufw/ufw-init start quiet (code=exited, status=0/SUCCESS)
  Main PID: 620 (code=exited, status=0/SUCCESS)
    CPU: 125ms

Mar 07 10:32:09 ubuntuuser systemd[1]: Starting ufw.service - Uncomplicated firewall...
Mar 07 10:32:09 ubuntuuser systemd[1]: Finished ufw.service - Uncomplicated firewall.
ozone@ubuntuuser:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-03-07 10:32:11 -03; 2min 24s ago
     Docs: man:fail2ban(1)
  Main PID: 940 (fail2ban-server)
    Tasks: 5 (limit: 4543)
   Memory: 55.2M (peak: 55.7M)
      CPU: 923ms
   CGroup: /system.slice/fail2ban.service
           └─940 /usr/bin/python3 /usr/bin/fail2ban-server -xf start
```

2. UFW. Reiniciar configuración del firewall.

Si bien en el anterior laboratorio ya configuramos **ufw**, en esta ocasión haremos todo desde 0. Hay que tener muy en claro que el resetear **NO** se debe aplicar a un servidor “vivo” (que está en producción), ya que en ese caso lo mejor es aplicar un enfoque de “cirujano”, esto quiere decir auditar y ajustar individualmente:

- **Verificar reglas existentes:** `sudo ufw status numbered` (esto da una lista numerada).
- **Eliminar lo que sobra:** `sudo ufw delete [número]`.
- **Asegurar las políticas por defecto:** Como ya habíamos visto en el Lab 3 “*Hardening Básico de Linux*”, podemos ejecutar `sudo ufw default deny incoming` (recordemos que dicha regla es la aplicación del concepto de mínimo privilegio) sin necesidad de resetear. Esto no borrará nuestras reglas de “*allow ssh*”, simplemente le dirá al firewall: *“Si no hay una regla específica que diga ‘pasa’, entonces bloquéalo”*.

Por lo tanto y habiendo dejado ya en claro cuando no conviene aplicar un reset, ahora dejamos en claro cuando si puede ser útil:

- **Configuración desde cero:** Es el estándar de “oro” al recibir un VPS nuevo para asegurarnos de que no traiga basura de configuraciones anteriores.
- **El firewall es un caos:** Si las reglas se han vuelto contradictorias o difíciles de auditar y preferimos reconstruir la seguridad de forma limpia y ordenada.

Para llevarlo a cabo:

```
sudo ufw reset
```

```
ozone@ubuntuerver:~$ sudo ufw reset
[sudo] password for ozone:
Resetting all rules to installed defaults. This may disrupt existing ssh
connections. Proceed with operation (y|n)? y
Backing up 'user.rules' to '/etc/ufw/user.rules.20260307_110826'
Backing up 'before.rules' to '/etc/ufw/before.rules.20260307_110826'
Backing up 'after.rules' to '/etc/ufw/after.rules.20260307_110826'
Backing up 'user6.rules' to '/etc/ufw/user6.rules.20260307_110826'
Backing up 'before6.rules' to '/etc/ufw/before6.rules.20260307_110826'
Backing up 'after6.rules' to '/etc/ufw/after6.rules.20260307_110826'
```


3. UFW. Aplicar política segura (principio de mínimo privilegio).

Esto ya lo habíamos hecho en el lab 3, en este lab vamos un poco más rápido con lo que ya se ha explicado previamente.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

Significa:

- todo tráfico entrante bloqueado
- solo se abre lo necesario

```
ozone@ubuntuuserver:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
ozone@ubuntuuserver:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
ozone@ubuntuuserver:~$
```

Nota: Se habrá notado como repito lo del “principio del mínimo privilegio”, y es que en Ciberseguridad es un concepto base que aparece en muchos textos y en diversos cursos, pero si bien se explica casi por sí solo, la aplicación del “deny incoming” es una muestra practica de dicho principio, es una forma de decir, así se aplica entre otras cosas como políticas de acceso y muchos ejemplos más para que solo el personal autorizado tenga acceso a la información, y así comienza a dialogar con la triada CIA, Confidencialidad, Integridad y Disponibilidad.

4. UFW. Abrir servicios necesarios.

Ejemplo básico.

SSH

```
sudo ufw allow 22/tcp
```

Web

```
sudo ufw allow 80/tcp
```

```
sudo ufw allow 443/tcp
```

```
ozone@ubuntuerver:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
ozone@ubuntuerver:~$ sudo ufw allow 80/tcp
Rules updated
Rules updated (v6)
ozone@ubuntuerver:~$ sudo ufw allow 443/tcp
Rules updated
Rules updated (v6)
```

¿No decían que http es inseguro? ¿Es una buena práctica abrir http y https?

Es una **excelente práctica**, pero con un matiz importante sobre la seguridad moderna.

¿Por qué abrir ambos?

- **Puerto 443 (HTTPS)**

Es **obligatorio**. Es el estándar actual para tráfico cifrado. Sin esto, los navegadores marcarán nuestro sitio como "No seguro" y los datos viajarán en texto plano.

- **Puerto 80 (HTTP)**

Se abre principalmente por **redirección**. Aunque no deseamos que la gente navegue en HTTP, necesitamos que el puerto esté abierto para que, cuando alguien escriba "nuestraweb.com" (que por defecto intenta entrar por el 80), nuestro servidor pueda decirle: "Oye, muévete al 443 (HTTPS)".

5. UFW. Limitar ataques SSH.

```
sudo ufw limit ssh
```

Este comando es una de las herramientas más sencillas y efectivas para frenar ataques de fuerza bruta. En lugar de simplemente dejar la puerta abierta (allow), el comando **limit** añade una capa de inteligencia:

- **Vigila la frecuencia:** Permite conexiones normales, pero si una misma dirección IP intenta conectarse **6 o más veces en menos de 30 segundos**, el firewall la bloquea automáticamente.
- **Deniega el acceso:** Esa IP queda "baneada" temporalmente, impidiendo que un bot pruebe miles de contraseñas por segundo.



```
ozone@ubuntuserver:~$ sudo ufw limit ssh
[sudo] password for ozone:
Rules updated
Rules updated (v6)
```


6. UFW. Activar firewall.

Importante: No hagas este paso si antes no permitiste el tráfico por el puerto 22 (ssh) del paso 3, de otro modo, te desconectará y te quedarás fuera (el famoso *lockout*).

```
sudo ufw enable
```

```
ozone@ubuntuuserver:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
ozone@ubuntuuserver:~$
```

Ver reglas numeradas.

```
sudo ufw status numbered
```

```
ozone@ubuntuuserver:~$ sudo ufw status numbered
Status: active
```

	To	Action	From
	--	-----	----
[1]	22/tcp	LIMIT IN	Anywhere
[2]	80/tcp	ALLOW IN	Anywhere
[3]	443/tcp	ALLOW IN	Anywhere
[4]	22/tcp (v6)	LIMIT IN	Anywhere (v6)
[5]	80/tcp (v6)	ALLOW IN	Anywhere (v6)
[6]	443/tcp (v6)	ALLOW IN	Anywhere (v6)

Ver configuración completa.

```
sudo ufw status verbose
```

```
ozone@ubuntuuserver:~$ sudo ufw status verbose
Status: active
Logging: on (high)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

To	Action	From
--	-----	----
22/tcp	LIMIT IN	Anywhere
80/tcp	ALLOW IN	Anywhere
443/tcp	ALLOW IN	Anywhere
22/tcp (v6)	LIMIT IN	Anywhere (v6)
80/tcp (v6)	ALLOW IN	Anywhere (v6)
443/tcp (v6)	ALLOW IN	Anywhere (v6)

6.1 Limitar ICMP.

```
sudo nano /etc/ufw/before.rules
```

Una buena práctica en la hardenización es limitar **ICMP** (algunos prefieren bloquear o botar simplemente los paquetes ICMP, utilizados por atacantes en escaneos de red o ataques de tipo *PING FLOOD*).

Como muestra la imagen a continuación, cuando ejecutamos

```
sudo nano /etc/ufw/before.rules
```

podemos ver que las reglas ICMP de UFW por defecto está configurada para **ACCEPT**. Esto es útil cuando necesitamos utilizar la herramienta ping para comprobar conectividad.

```
# ok icmp codes for INPUT
-A ufw-before-input -p icmp --icmp-type destination-unreachable -j ACCEPT
-A ufw-before-input -p icmp --icmp-type time-exceeded -j ACCEPT
-A ufw-before-input -p icmp --icmp-type parameter-problem -j ACCEPT
-A ufw-before-input -p icmp --icmp-type echo-request -j ACCEPT
# ok icmp code for FORWARD
```

Como dije previamente algunos prefieren aplicar reglas como las siguientes que funcionan bien frente a escaneos de red.

Cambiamos **ACCEPT** por **DROP**

```
# ok icmp codes for INPUT
-A ufw-before-input -p icmp --icmp-type destination-unreachable -j ACCEPT
-A ufw-before-input -p icmp --icmp-type time-exceeded -j ACCEPT
-A ufw-before-input -p icmp --icmp-type parameter-problem -j ACCEPT
-A ufw-before-input -p icmp --icmp-type echo-request -j DROP
```

o simplemente podemos comentar la línea con #

```
# ok icmp codes for INPUT
-A ufw-before-input -p icmp --icmp-type destination-unreachable -j ACCEPT
-A ufw-before-input -p icmp --icmp-type time-exceeded -j ACCEPT
-A ufw-before-input -p icmp --icmp-type parameter-problem -j ACCEPT
#-A ufw-before-input -p icmp --icmp-type echo-request -j ACCEPT
```

El problema con esto es que puede traer algunos problemas y dolores de cabeza asociados por lo que es necesario estar claros en que estrategia es la que deberíamos utilizar.

1. DROP vs. REJECT

Cuando un paquete de "ping" llega a nuestro servidor y el firewall tiene una regla aplicada, puede reaccionar de dos formas:

- **DROP**

El firewall ignora el paquete. El que envió el ping se queda esperando (hace *timeout*) hasta que se cansa.

Problema: Los escáneres de red modernos detectan este silencio como *"aquí hay algo que me está bloqueando deliberadamente"*. Además, dificulta mucho el diagnóstico de red porque no sabemos si el servidor está caído o si el firewall está filtrando.

- **REJECT**

El firewall responde activamente con un mensaje de *"Destino inalcanzable"*.

Ventaja: Las aplicaciones de red (como Wazuh, CrowdSec) y las herramientas de diagnóstico reciben una respuesta inmediata de que el puerto está cerrado, evitando esperas innecesarias y comportamientos erráticos en los timeouts.

2. Rate-Limiting (El punto medio inteligente)

Esta es la mejor opción. Permitimos el ping, pero limitamos **cuántos** puede recibir el servidor por segundo. Esto evita que alguien sature nuestra CPU o nuestro ancho de banda con un ataque de inundación (Ping Flood), pero permite que nuestras herramientas de monitoreo sigan viendo que el servidor está vivo.

¿Qué pasará con nuestras aplicaciones?

- **Wazuh:** Si bien aún no lo hemos instalado, cuando lo hagamos seguirá viendo el agente "online". Wazuh no ametralla el servidor con pings; suele enviar paquetes de control (keep-alive) con intervalos mayores a 2 segundos. Estamos a salvo.
- **CrowdSec:** Funcionará mejor. Si un atacante lanza un escaneo de red (`nmap -sn`), los primeros 3 pings responderán, pero el resto fallará. Esto es comportamiento sospechoso que CrowdSec puede registrar y usar para banear la IP por "escaneo de puertos/red".
- **Docker:** Si bien lo instalaremos más adelante, probablemente con Wazuh, los contenedores que necesiten verificar conectividad externa lo harán sin problemas, ya que raramente una aplicación legítima necesita hacer 10 pings por segundo.
- **Pruebas de conexión:** Si intentamos hacer un ping manual desde nuestra PC, veremos que los primeros 3 responden instantáneamente y luego veremos que los siguientes tardan o se pierden si no esperamos los 2 segundos. **Esto es normal y significa que la regla funciona.**

5.1.1 Añadir excepciones a la regla.

Debido a que estamos empezando a aplicar reglas restrictivas, es recomendable añadir excepciones antes de comenzar a realizar pruebas con el objeto de evitar el desagradable "lockout" y que por error nuestra maquina Ubuntu, nos bloquee la ip y ya no podamos ingresar; en este caso añadiremos la dirección ip de nuestro equipo anfitrión, en mi caso Windows. No añadiré la de Kali por que la idea es practicar como si fuese un entorno real y podríamos obtener resultados falsos si aplico dicha excepción.

¿Cómo hacerlo?

El orden de las reglas es muy importante, lo que haremos es colocar la regla de confianza **ANTES** de la regla del limit.

En el archivo **/etc/ufw/before.rules**, añadir esto justo arriba de la línea del limit:

Permitir ping ilimitado solo desde mi PC Windows

```
-A ufw-before-input -p icmp --icmp-type echo-request -s 192.168.0.XX -j ACCEPT
```

Regla restrictiva para el resto del mundo

```
-A ufw-before-input -p icmp --icmp-type echo-request -m limit --limit 30/minute --limit-burst 3 -j ACCEPT
```

(Sustituye 192.168.0.XX por la IP fija de tu Windows).

```
-A ufw-before-input -m conntrack --ctstate INVALID -j ufw-logging-deny
-A ufw-before-input -m conntrack --ctstate INVALID -j DROP

# Permitir ping ilimitado solo desde mi PC Windows
-A ufw-before-input -p icmp --icmp-type echo-request -s 192.168.0.32 -j ACCEPT

# ok icmp codes for INPUT
-A ufw-before-input -p icmp --icmp-type destination-unreachable -j ACCEPT
-A ufw-before-input -p icmp --icmp-type time-exceeded -j ACCEPT
-A ufw-before-input -p icmp --icmp-type parameter-problem -j ACCEPT
-A ufw-before-input -p icmp --icmp-type echo-request -m limit --limit 30/minute --limit-burst 3 -j ACCEPT

# ok icmp code for FORWARD
-A ufw-before-forward -p icmp --icmp-type destination-unreachable -j ACCEPT
```

- **--limit 30/minute:** Permite exactamente un paquete cada 2 segundos. Cualquier intento más rápido que eso será ignorado (o rechazado, según el resto de nuestras reglas).
- **--limit-burst 3:** Es el "token" inicial. Permite que los primeros 3 pings pasen rápido (por si estamos haciendo una prueba rápida de conexión), pero una vez gastados esos 3, entra en vigor el límite estricto de 1 cada 2 segundos.

Guardamos y reiniciamos ufw.

```
sudo ufw reload
```

7. Activar logs detallados.

Si bien ya está activado y configurado en HIGH como se puede ver en la captura de pantalla anterior, en este punto creo que es importante mostrar la diferencia entre los niveles.

Lo primero es señalar que UFW tiene varios niveles de registro (*low, medium, high, full*).

Lo segundo, no tiene que ser obligatoriamente en high, pero se recomienda ese nivel cuando estamos configurando o auditando la seguridad de un servidor por primera vez.

Nivel	Qué registra	¿Cuándo usarlo?
Low	Solo paquetes bloqueados que no coinciden con tus reglas (el mínimo).	Servidores con recursos muy limitados.
Medium	Igual que Low , pero añade paquetes bloqueados que no coinciden con la política por defecto, además de intentos de conexión nuevos.	Recomendado: Buen equilibrio entre info y espacio en disco.
High	Registra todo lo anterior más los paquetes que no tienen una regla específica de "limit".	Para auditorías de seguridad o si hay sospechas de un ataque.
Full	Registra absolutamente todo el tráfico (aceptado y denegado), sin ninguna restricción.	Solo para diagnósticos breves. Puede llenar el disco en horas.

Yo personalmente lo dejaré en HIGH, debido a que como en este caso estamos en un Lab en el que más adelante lanzaremos ataques desde Kali y además el espacio en disco es de 30GB (ese espacio lo asigne en el Lab1 "Instalación y preparación de Ubuntu Server"); se puede confirmar con el comando **df**.

```
ozone@ubuntuuserver:~$ df
Filesystem            1K-blocks    Used Available Use% Mounted on
tmpfs                  396088      1524    394564   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 14339080 6339484  7249416  47% /
tmpfs                  1980436        0    1980436   0% /dev/shm
tmpfs                   5120         0      5120    0% /run/lock
/dev/sda2              1992552    202512    1668800  11% /boot
tmpfs                  396084        12    396072   1% /run/user/1000
```

Al revisar, me doy cuenta que cuando se realizó la instalación, LVM (Logical Volume Manager) solo tomo la mitad del espacio disponible.

7.1. Expandir espacio de disco (lsblk).

En caso de que Ubuntu no nos reconozca el tamaño total asignado a la maquina en VMWare o Virtual Box simplemente haremos lo siguiente:

1. **Verificar tamaño:** Con el comando `lsblk` verificamos el tamaño del disco.

```
ozone@ubuntu-server:~$ lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                8:0    0   30G  0 disk
├─sda1                             8:1    0    1M  0 part
├─sda2                             8:2    0    2G  0 part /boot
├─sda3                             8:3    0   28G  0 part
└─ubuntu--vg-ubuntu--lv 252:0    0   14G  0 lvm /
sr0                               11:0    1 1024M  0 rom
```

Como se puede ver `lsblk` confirma que el disco **sda** es de **30G** pero mi **LV** es de **14G**, por lo tanto utilizaré los siguientes comandos en orden. Son seguros y se pueden hacer con el sistema encendido.

2. **Confirmar nombre del volumen lógico:**

```
sudo lvdisplay
```

```
ozone@ubuntu-server:~$ sudo lvdisplay
[sudo] password for ozone:
--- Logical volume ---
LV Path                /dev/ubuntu-vg/ubuntu-lv
LV Name                 ubuntu-lv
VG Name                 ubuntu-vg
LV UUID                 -LgK2-cB49-FnWr-L1NElX
LV Write Access         read/write
LV Creation host, time ubuntu-server, 2026-02-09 17:52:01 -0300
LV Status                available
# open                  1
LV Size                 <14.00 GiB
Current LE               3583
Segments                1
Allocation               inherit
Read ahead sectors      auto
- currently set to      256
Block device            252:0
```

3. Extender el volumen lógico al máximo espacio libre:

```
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
```

```
ozone@ubuntu-server:~$ sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
Size of logical volume ubuntu-vg/ubuntu-lv changed from <14.00 GiB (3583 extents) to <28.00 GiB (7167 extents).
Logical volume ubuntu-vg/ubuntu-lv successfully resized.
```

4. Informar al sistema de archivos del nuevo tamaño:

```
sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
```

```
ozone@ubuntu-server:~$ sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/mapper/ubuntu--vg-ubuntu--lv is mounted on /; on-line resizing required
old_desc_blocks = 2, new_desc_blocks = 4
The filesystem on /dev/mapper/ubuntu--vg-ubuntu--lv is now 7339008 (4k) blocks long.
```

5. Confirmar el nuevo tamaño:

```
df -h
```

```
ozone@ubuntu-server:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	387M	1.5M	386M	1%	/run
/dev/mapper/ubuntu--vg-ubuntu--lv	28G	6.1G	21G	24%	/
tmpfs	1.9G	0	1.9G	0%	/dev/shm
tmpfs	5.0M	0	5.0M	0%	/run/lock
/dev/sda2	2.0G	198M	1.6G	11%	/boot
tmpfs	387M	12K	387M	1%	/run/user/1000

```
ozone@ubuntu-server:~$
```

Y como se puede ver ya tenemos asignados el tamaño total por lo que ahora podemos colocar el logging en high sin miedo a saturar nuestro disco.

7.2 Configurar logs detallados.

Entonces, ahora podemos retomar y configurar el tipo de *ufw logging*. En mi caso, high:

```
sudo ufw logging high
```

```
ozone@ubuntuuserver:~$ sudo ufw logging high
[sudo] password for ozone:
Logging enabled
```

Log principal:

```
/var/log/ufw.log | grep
```

Aquí veremos:

- Escaneos.
- Conexiones rechazadas.
- Intentos sospechosos.

Por ejemplo si filtramos por conexiones rechazadas:

```
ozone@ubuntuuserver:~$ sudo cat /var/log/ufw.log | grep "BLOCK"
2026-03-05T18:33:38.445486-03:00 ubuntuuserver kernel: [UFW BLOCK] IN=ens33 OUT= MAC=00
:0c: :a3:00: :ca:01: :03: :00 SRC=192.168.0.252 DST=192.168.0.36 LEN=71 TOS=
0x00 PREC=0x00 TTL=64 ID=0 DF PROTO=UDP SPT=39129 DPT=5353 LEN=51
```

podemos ver que rechaza las conexiones al puerto 5353, esto sucede porque en el paso 3 aplicamos:

```
sudo ufw default deny incoming
```

y no tenemos una regla específica que permita el tráfico en el puerto **5353** (usado comúnmente por el protocolo *mDNS/Avahi* para "descubrimiento de servicios" en redes locales).

8. Fail2Ban Configuración.

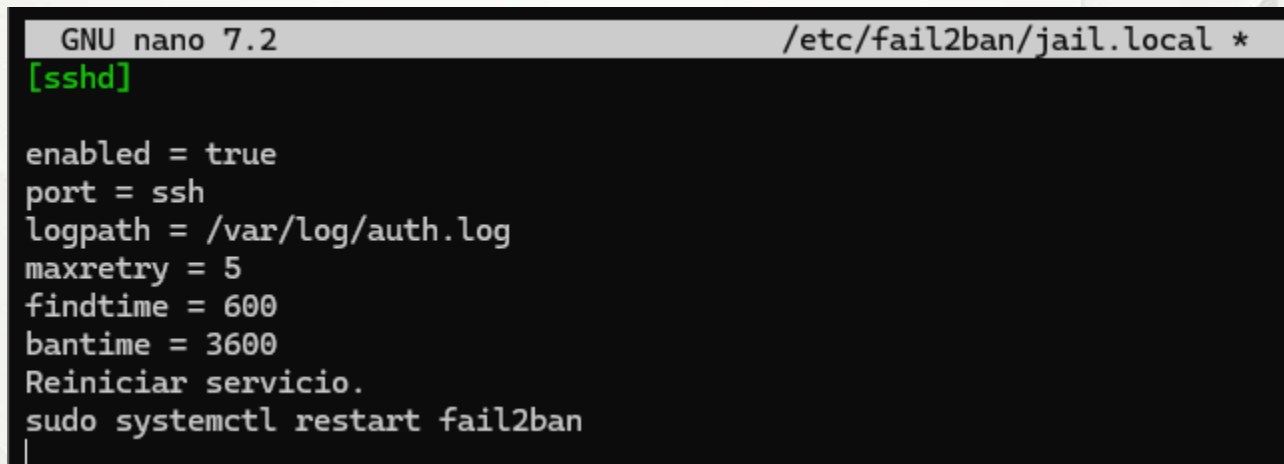
Crear configuración local.

```
sudo nano /etc/fail2ban/jail.local
```

Configuración SSH.

```
[sshd]

enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 5
findtime = 600
bantime = 3600
```



```
GNU nano 7.2 /etc/fail2ban/jail.local *
[sshd]

enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 5
findtime = 600
bantime = 3600
Reiniciar servicio.
sudo systemctl restart fail2ban
```

Reiniciar servicio.

```
sudo systemctl restart fail2ban
```

Es necesario entender lo que estamos ingresando para cuando necesitemos adaptar a otras configuraciones:

Parámetro	Valor	Descripción
[sshd]	Sección	Define que estas reglas se aplican solo al servicio SSH .
enabled	true	Activa la protección para este servicio específico.
port	ssh	Indica que debe vigilar el puerto 22 (o el puerto configurado).
logpath	/var/log/auth.log	El archivo de sistema donde Fail2Ban lee los intentos de acceso.
maxretry	5	Número de intentos fallidos permitidos antes del baneo.
findtime	600	Ventana de tiempo (10 min) en la que deben ocurrir los fallos.
bantime	3600	Tiempo que la IP atacante estará bloqueada (1 hora).



9. Fail2ban. Ver IPs bloqueadas.

Estado general.

```
sudo fail2ban-client status
```

Estado SSH.

```
sudo fail2ban-client status sshd
```

```
ozone@ubuntu-server:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
`- Jail list:  sshd
ozone@ubuntu-server:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:    0
|  `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:    0
   `-- Banned IP list:
ozone@ubuntu-server:~$ |
```

La captura indica que todo está funcionando normalmente.

Como aún no he generado ataques de fuerza bruta los filtros aparecen en 0.

9.1 Prueba de fuerza bruta con Hydra

Para comprobar que todo está funcionando, realizaremos un par de ataques de fuerza bruta desde Kali con Hydra.

```
hydra -l usuario_falso -p contrasenia123 192.168.0.36 ssh
```

```
hydra -l usuario_falso -p contrasena456 192.168.0.36 ssh
```

```
(ozone@kali)-[~]
$ hydra -l usuario_falso -p contrasenia123 192.168.0.36 ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-07 15:32:17
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking ssh://192.168.0.36:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-07 15:32:20

(ozone@kali)-[~]
$ hydra -l usuario_falso -p contrasena456 192.168.0.36 ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-07 15:36:08
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking ssh://192.168.0.36:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-07 15:36:13
```

Y si revisamos el status de fail2ban en nuestra VM Ubuntu podemos ver como aparecen los intentos del primer ataque.

```
ozone@ubuntuuserver:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 1
|   |- Total failed: 3
|   \- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
\-- Actions
    |- Currently banned: 0
    |- Total banned: 0
    \- Banned IP list:
```


¿Por qué tuvieron que ser 2 ataques con Hydra?

De acuerdo a lo que se ve en la salida como muestra la imagen anterior es que Fail2Ban ya detectó los intentos, pero todavía no ha "disparado" el baneo porque configuramos el límite en 5 (maxretry = 5). En la salida dice "Total failed: 3". Eso significa que a nuestra máquina Kali le quedan **2 intentos más** antes de que su IP aparezca en la lista negra. Por este motivo se lanzó el segundo ataque y al consultar nuevamente la salida que podemos apreciar en la imagen siguiente, nos indica que hay una ip baneada e incluso nos dice cual es.

```
ozone@ubuntu-server:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 1
|   |- Total failed: 6
|   \- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
\-- Actions
    |- Currently banned: 1
    |- Total banned: 1
    \- Banned IP list: 192.168.0.38
```

9.2 Desbloquear una IP bloqueada

En el caso que necesitemos desbloquear una ip, lo hacemos con:

```
sudo fail2ban-client set sshd unbanip [IP_a_desbloquear]
```

```
ozone@ubuntu-server:~$ sudo fail2ban-client set sshd unbanip 192.168.0.38
1
```

y verificamos nuevamente con

```
sudo fail2ban-client status sshd
```

```
ozone@ubuntu-server:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 1
|   |- Total failed: 6
|   \- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
\-- Actions
    |- Currently banned: 0
    |- Total banned: 1
    \- Banned IP list:
```

10. Fail2ban. Bloqueo agresivo contra fuerza bruta.

Abriremos el archivo que creamos previamente

```
sudo nano /etc/fail2ban/jail.local
```

Agregaremos este jail adicional.

```
[sshd-aggressive]

enabled = true
port = ssh
filter = sshd[mode=aggressive]
logpath = /var/log/auth.log
maxretry = 3
findtime = 300
bantime = 86400
```

```
[sshd]
enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 5
findtime = 600
bantime = 3600

[sshd-aggressive]
enabled = true
port = ssh
filter = sshd[mode=aggressive]
logpath = /var/log/auth.log
maxretry = 3
findtime = 300
bantime = 86400
```

Reiniciamos el servicio para que los cambios surjan efecto.

```
sudo systemctl restart fail2ban
```

Al consultar el estado del cliente fail2ban podemos ver que nos muestra ahora 2 jaulas.

```
sudo fail2ban-client status
```

```
ozone@ubuntu-server:~$ sudo fail2ban-client status
Status
|- Number of jail:      2
`- Jail list:  sshd, sshd-aggressive
```

Para ver el estado de esta nueva jaula filtramos por su nombre:

```
sudo fail2ban-client status sshd-aggressive
```

```
ozone@ubuntu-server:~$ sudo fail2ban-client status sshd-aggressive
Status for the jail: sshd-aggressive
|- Filter
| |- Currently failed: 0
| |- Total failed:     0
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
  |- Currently banned: 0
  |- Total banned:     0
  `-- Banned IP list:
```

¿Por qué tener los dos o más?

Lo que estamos haciendo es crear dos redes (trampas) de seguridad con diferentes "agujeros":

1. **[sshd]**: Es la red permisiva. Perdona más fallos (5) y el castigo es corto (1 hora). Es para usuarios humanos que se equivocan de contraseña.
2. **[sshd-aggressive]**: Es la red para bots. Si alguien falla 3 veces en solo 5 minutos, Fail2Ban asume que no es un humano despistado, sino un ataque automatizado (como Hydra), y lo bloquea por **24 horas** (86400 segundos).

11. Fail2ban. Detectar portscan.

Activaremos una capa de protección contra el escaneo de puertos. Es importante entender la diferencia con las anteriores

La diferencia principal con las 2 anteriores radica en **QUÉ** está intentando detectar el sistema y **DÓNDE** está mirando para encontrarlo. Aunque ambas son capas de seguridad, operan en niveles distintos.

Aquí tenemos la comparativa detallada:

1. El objetivo (¿Qué bloquean?)

- **[portscan]**: Es una defensa **perimetral**. Bloquea a alguien que está "tanteando" nuestro servidor para ver qué puertos están abiertos (usando herramientas como Nmap). Detecta el intento antes siquiera de que el atacante intente loguearse.
- **[sshd-aggressive]**: Es una defensa **específica de aplicación**. Bloquea a alguien que ya encontró el puerto SSH y está intentando entrar por la fuerza, ya sea con contraseñas erróneas o aprovechando vulnerabilidades del protocolo.

2. Fuente de información (Logs)

Fail2Ban es como un detective que lee diarios (logs). Cada "jail" lee un diario distinto:

Característica	[portscan]	[sshd-aggressive]
Log que lee	/var/log/ufw.log	/var/log/auth.log
Capa	Red/Firewall (UFW)	Aplicación (Servicio SSH)
Evento clave	Paquetes rechazados por el firewall.	Intentos de login fallidos o errores de conexión SSH.

3. Agresividad y severidad

Si nos fijamos en los parámetros de ambos:

- **Dureza del castigo (bantime)**
 - El de SSH es mucho más severo: **86400s (24 horas)** vs los **3600s (1 hora)** del escáner de puertos. Esto es porque un ataque de fuerza bruta al login se considera una amenaza más directa que un simple escaneo.
- **Sensibilidad (maxretry)**
 - El de SSH salta con solo **3 intentos**, mientras que el escáner de puertos da **10 oportunidades** antes de banear.



11.1 Agregar reglas para detección de escaneo de puertos.

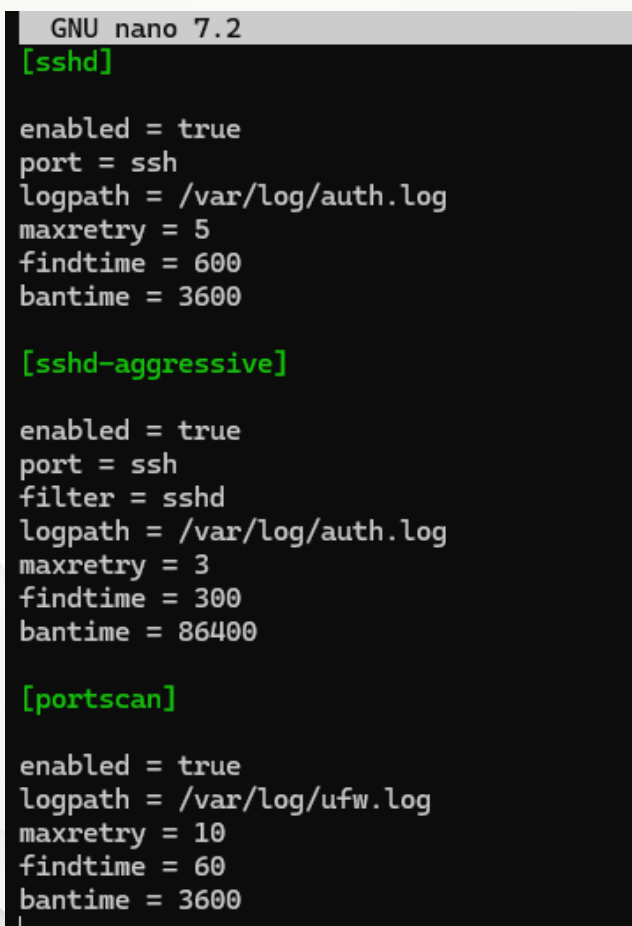
Abriremos el archivo que creamos previamente

```
sudo nano /etc/fail2ban/jail.local
```

Agregaremos este jail adicional.

```
[portscan]

enabled = true
logpath = /var/log/ufw.log
maxretry = 10
findtime = 60
bantime = 3600
```



```
GNU nano 7.2
[sshd]

enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 5
findtime = 600
bantime = 3600

[sshd-aggressive]

enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
findtime = 300
bantime = 86400

[portscan]

enabled = true
logpath = /var/log/ufw.log
maxretry = 10
findtime = 60
bantime = 3600
```

Guardamos y reiniciamos el servicio para que los cambios surjan efecto.

```
sudo systemctl restart fail2ban
```

12. Fail2ban. Bloquear IP reincidentes (recidive)

Fail2Ban incluye un sistema llamado **recidive**. Esta es, posiblemente, la configuración más importante para mantener la salud de nuestro servidor a largo plazo. El módulo [recidive] es el "calabozo de máxima seguridad" de Fail2Ban.

La forma de agregarlo es la misma que los anteriores.

```
[recidive]

enabled = true
logpath = /var/log/fail2ban.log
bantime = 604800
findtime = 86400
maxretry = 5
```

Si una IP es bloqueada muchas veces el bloqueo será por **7 días (604800 seg)**.

Para consultar su estado, recordamos que lo hacemos agregando el nombre de la "jaula".

```
sudo fail2ban-client status recidive
```

12.1 IMPORTANTE. Ignorar direcciones IP.

El módulo recidive es muy potente, pero **no perdona**.

Si nos equivocamos de contraseña de SSH varias veces y nos banea el módulo [sshd], podríamos entrar en la lista de recidive si insistes. Una vez que entres ahí, estaremos bloqueado por una semana completa.

Antes de activar recidive, debemos asegurarnos de agregar aquellas direcciones IP nuestra propia IP (o la red de nuestra casa/oficina), como también la de nuestra maquina Kali a la lista de ignorados en la sección [DEFAULT] de nuestro archivo jail.local.

```
[DEFAULT]
ignoreip = <direcciones ip a ignorar>
```

```
GNU nano 7.2 /etc/fail2ban/jail.local *
[DEFAULT]
ignoreip = 127.0.0.1/8 ::1 192.168.0.32 192.168.0.36 192.168.0.38

[sshd]

enabled = true
port = ssh
```

Básicamente lo que hacemos es ingresar direcciones en una **“lista de confianza”**

Dirección / Rango	Tipo de protocolo	Descripción y uso
127.0.0.1/8	IPv4 (Local)	Es la dirección de localhost . Permite que el servidor se comunique con sus propios servicios internos sin riesgo de auto-bloqueo.
::1	IPv6 (Local)	Es el equivalente exacto de localhost pero para redes IPv6 . Es fundamental incluirlo en sistemas modernos.
192.168.x.x	IPv4 (Privada)	Representa las direcciones de tu red local (LAN) . Aquí incluyes tu PC de trabajo, tu máquina de Kali o cualquier equipo en el mismo laboratorio.
IP Pública	IPv4 (Externa)	Direcciones fijas de internet (ej. 200.x.x.x). Se agregan para dar acceso permanente a administradores que se conectan desde fuera de la red local.

13. Fail2ban. Detectar IP que escanean muchos puertos

Esta regla es como una versión "reforzada" de la regla portscan.

Mientras que portscan detecta a alguien buscando qué puertos tenemos abiertas, el **port-hammer** (martilleo de puertos) castiga a quien golpea nuestros puertos de forma masiva en muy poco tiempo, aplicando un baneo mucho más largo (24 horas en lugar de 1 hora).

Jail	Maxretry	Bantime	Objetivo
[portscan]	10 intentos	1 hora	Detectar escaneos rápidos de reconocimiento.
[port-hammer]	15 intentos	24 horas	Castigar ataques de fuerza bruta o escaneos masivos persistentes.

¿Hay algún riesgo de tener ambas?

No hay riesgo de "romper" el sistema, pero debemos tener en cuenta esto:

Como ambos leen el mismo archivo (/var/log/ufw.log), si un atacante hace un escaneo rápido, **primero saltará el baneo de portscan** (porque requiere menos reintentos, solo 10).

Sin embargo, si la IP sigue atacando después de que expire esa hora, o si el ataque es tan masivo que genera 15 logs casi instantáneamente, entrará en juego el **port-hammer** y lo dejará fuera por un día completo.

Es como tener una alarma que suena a los 10 segundos y un guardia que llega a los 15.

Agregar jail.

```
[port-hammer]
```

```
enabled = true
logpath = /var/log/ufw.log
maxretry = 15
findtime = 60
bantime = 86400
```

Nuestro archivo de “jaulas” de fail2ban configurado:

```
GNU nano 7.2 /etc/fail2ban/jail.local *
[DEFAULT]
ignoreip = 127.0.0.1/8 ::1 192.168.0.32 192.168.0.36 192.168.0.38

[sshd]
enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 5
findtime = 600
bantime = 3600

[sshd-aggressive]
enabled = true
port = ssh
filter = sshd[mode=aggressive]
logpath = /var/log/auth.log
maxretry = 3
findtime = 300
bantime = 86400

[portscan]
enabled = true
logpath = /var/log/ufw.log
maxretry = 10
findtime = 60
bantime = 3600

[port-hammer]
enabled = true
logpath = /var/log/ufw.log
maxretry = 15
findtime = 60
bantime = 86400

[recidive]
enabled = true
logpath = /var/log/fail2ban.log
bantime = 604800
findtime = 86400
maxretry = 5
```

14. UFW. Protección contra SYN flood

La regla que agregaremos sirve para limitar la tasa de nuevas conexiones TCP (mitigación de ataques tipo **SYN Flood**).

En el archivo **before.rules**, el orden es fundamental porque las reglas se procesan de arriba hacia abajo. La ubicación ideal es justo **antes** de que empiecen las reglas específicas de protocolos (como ICMP o DHCP), pero **después** de permitir el tráfico de loopback y las conexiones ya establecidas.

¿Por qué ahí?

1. **Después de ESTABLISHED**: No queremos limitar la velocidad de las conexiones que ya son legítimas y están abiertas; solo queremos limitar los **nuevos** intentos (--syn).
2. **Antes de las reglas de bloqueo**: Esto actúa como un "filtro de entrada" que calma el flujo de paquetes antes de que lleguen a otras capas de procesamiento.

Explicación técnica de la regla:

- **--limit 10/s**
Permite un promedio de 10 nuevas conexiones por segundo.
- **--limit-burst 20**
Permite un "pico" inicial de hasta 20 conexiones antes de empezar a aplicar el límite de 10/s.

Aplicación:

```
sudo nano /etc/uw/before.rules
```

Agregar limite.

```
-A uw-before-input -p tcp --syn -m limit --limit 10/s --limit-burst 20 -j ACCEPT
```

Debería verse como la siguiente imagen:

```
GNU nano 7.2 /etc/ufw/before.rules *
#
# Don't delete these required lines, otherwise there will be errors
*filter
:ufw-before-input - [0:0]
:ufw-before-output - [0:0]
:ufw-before-forward - [0:0]
:ufw-not-local - [0:0]
# End required lines

# allow all on loopback
-A ufw-before-input -i lo -j ACCEPT
-A ufw-before-output -o lo -j ACCEPT

# quickly process packets for which we already have a connection
-A ufw-before-input -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A ufw-before-output -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A ufw-before-forward -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT

# Limitar nuevas conexiones TCP (SYN) para prevenir ataques DoS
-A ufw-before-input -p tcp --syn -m limit --limit 10/s --limit-burst 20 -j ACCEPT

# drop INVALID packets (logs these in loglevel medium and higher)
-A ufw-before-input -m conntrack --ctstate INVALID -j ufw-logging-deny
-A ufw-before-input -m conntrack --ctstate INVALID -j DROP

# ok icmp codes for INPUT
-A ufw-before-input -p icmp --icmp-type destination-unreachable -j ACCEPT
-A ufw-before-input -p icmp --icmp-type time-exceeded -j ACCEPT
-A ufw-before-input -p icmp --icmp-type parameter-problem -j ACCEPT
-A ufw-before-input -p icmp --icmp-type echo-request -j ACCEPT

# ok icmp code for FORWARD
-A ufw-before-forward -p icmp --icmp-type destination-unreachable -j ACCEPT
-A ufw-before-forward -p icmp --icmp-type time-exceeded -j ACCEPT
-A ufw-before-forward -p icmp --icmp-type parameter-problem -j ACCEPT
-A ufw-before-forward -p icmp --icmp-type echo-request -j ACCEPT

# allow dhcp client to work
-A ufw-before-input -p udp --sport 67 --dport 68 -j ACCEPT

#
# ufw-not-local
#
```


15. CrowdSec (IDS/IPS colaborativo)

Ahora que tenemos una defensa local sólida con **Fail2Ban**, vamos a añadir una **capa de inteligencia global con CrowdSec**. Mientras Fail2Ban vigila nuestros errores locales, CrowdSec nos protegerá contra atacantes ya identificados en todo el mundo.

CrowdSec es el sucesor espiritual de Fail2Ban. Mientras Fail2Ban actúa solo, CrowdSec usa **inteligencia colectiva**: si una IP ataca a un usuario en Japón, se bloquea automáticamente en tu servidor. Y muy importante para nuestro laboratorio, es Open Source.

¿Qué detecta por defecto?

Al instalarse, CrowdSec escanea los servicios y activa "colecciones" automáticamente para:

- **SSH Brute Force**: Intentos fallidos de login.
- **HTTP Scanners**: Bots buscando archivos **.env** o carpetas **/admin**.
- **IPs maliciosas**: Bloquea proactivamente las IPs de la lista comunitaria global.

****Tip Pro**: Podemos registrar nuestro servidor en app.crowdsec.net para tener un panel visual (Dashboard) gratuito de todos los ataques que recibe nuestra máquina.

Es como tener **threat intelligence gratis**.

Instalación.

```
curl -s https://install.crowdsec.net | sudo bash
```

```
ozone@ubuntuserver:~$ curl -s https://install.crowdsec.net | sudo bash
[sudo] password for ozone:
Detected operating system as ubuntu/24.
Detected apt version as 2.8.3
Checking for gpg...
Detected gpg...
Checking for curl...
Detected curl...
Scanning processes...
```

```
sudo apt install crowdsec -y
```

```
ozone@ubuntuserver:~$ sudo apt install crowdsec -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  crowdsec
0 upgraded, 1 newly installed, 0 to remove and 5 not upgraded.
Need to get 60.9 MB of archives.
After this operation, 226 MB of additional disk space will be used.
```

Instalación del Bouncer (El "brazo ejecutor")

El motor solo detecta; necesitamos un **bouncer** para **bloquear** el tráfico.

Aunque usemos UFW, se recomienda el bouncer de iptables o nftables porque actúa a un nivel más profundo del kernel.

Instalamos el bouncer:

```
sudo apt install crowdsec-firewall-bouncer-iptables -y
```

```
ozone@ubuntuserver:~$ sudo apt install crowdsec-firewall-bouncer-iptables -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ipset libipset13
The following NEW packages will be installed:
  crowdsec-firewall-bouncer-iptables ipset libipset13
0 upgraded, 3 newly installed, 0 to remove and 5 not upgraded.
Need to get 4,497 kB of archives.
```

Revisar estado:

```
sudo systemctl status crowdsec
```

```
ozone@ubuntuuserver:~$ sudo systemctl status crowdsec
● crowdsec.service - Crowdsec agent
   Loaded: loaded (/usr/lib/systemd/system/crowdsec.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-03-09 16:13:58 -03; 5min ago
     Main PID: 2785 (crowdsec)
        Tasks: 8 (limit: 4543)
      Memory: 23.3M (peak: 32.2M)
         CPU: 3.123s
       CGroup: /system.slice/crowdsec.service
              └─2785 /usr/bin/crowdsec -c /etc/crowdsec/config.yaml

Mar 09 16:13:55 ubuntuuserver systemd[1]: Starting crowdsec.service - Crowdsec agent...
Mar 09 16:13:58 ubuntuuserver systemd[1]: Started crowdsec.service - Crowdsec agent.
ozone@ubuntuuserver:~$ |
```

Comandos esenciales de gestión

Comando	Función
<code>sudo cscli metrics</code>	Ver estadísticas de detección y lectura de logs.
<code>sudo cscli decisions list</code>	Ver qué IPs están baneadas actualmente.
<code>sudo cscli bouncers list</code>	Verificar que el firewall esté conectado al motor.
<code>sudo cscli alerts list</code>	Historial de ataques detectados.

`sudo cscli metrics`

```
ozone@ubuntuuserver:~$ sudo cscli metrics
```

Acquisition Metrics					
Source whitelisted	Lines read	Lines parsed	Lines unparsed	Lines poured to bucket	Lines
file:/var/log/auth.log	10	-	10	-	-
file:/var/log/kern.log	54	-	54	-	-
file:/var/log/syslog	68	-	68	-	-

Local API Metrics		
Route	Method	Hits
/v1/decisions/stream	GET	8
/v1/heartbeat	GET	3
/v1/usage-metrics	POST	1
/v1/watchers/login	POST	1

`sudo cscli decisions list`

```
ozone@ubuntuuserver:~$ sudo cscli decisions list
No active decisions
ozone@ubuntuuserver:~$ |
```


`sudo cscli bouncers list`

```
ozone@ubuntuuserver:~$ sudo cscli bouncers list
```

Name	IP Address	Valid	Last API pull	Type	
ersion				Auth Type	V
cs-firewall-bouncer-177	49 127.0.0.1	✓	2026-03-09T19:23:57Z	crowdsec-firewall-bouncer	v
0.0.34-debian-pragmatic-amd64-41		!ae:		.98 api-key	

```
ozone@ubuntuuserver:~$ |
```

`sudo cscli alerts list`

```
ozone@ubuntuuserver:~$ sudo cscli alerts list
No active alerts
ozone@ubuntuuserver:~$ |
```



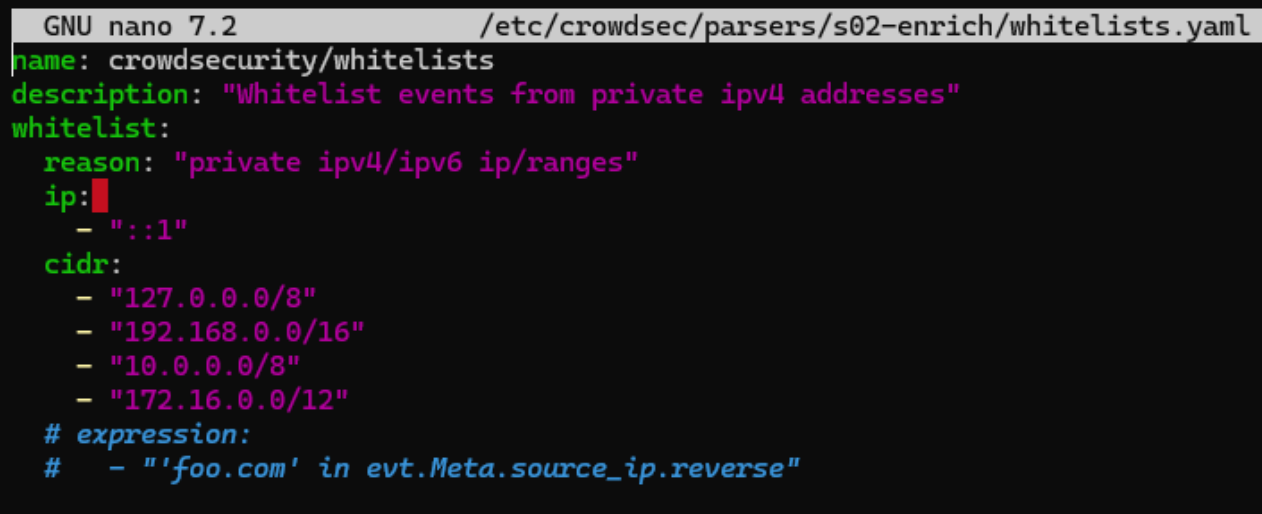
15.1 IMPORTANTE. Ignorar direcciones IP (Whitelist).

CrowdSec tiene su propia lista blanca. Si vamos a instalarlo, debemos añadir este paso para que no nos bloquee:

Editar el archivo de whitelist de CrowdSec:

```
sudo nano /etc/crowdsec/parsers/s02-enrich/whitelists.yaml
```

Cuando ingresamos por primera vez veremos que hay reglas por defecto:



```
GNU nano 7.2 /etc/crowdsec/parsers/s02-enrich/whitelists.yaml
name: crowdsecurity/whitelists
description: "Whitelist events from private ipv4 addresses"
whitelist:
  reason: "private ipv4/ipv6 ip/ranges"
  ip:
    - ":::1"
  cidr:
    - "127.0.0.0/8"
    - "192.168.0.0/16"
    - "10.0.0.0/8"
    - "172.16.0.0/12"
# expression:
# - "'foo.com' in evt.Meta.source_ip.reverse"
```

Borraremos el contenido y dejaremos lo siguiente:

```
name: crowdsecurity/whitelists
description: "Whitelist selectiva para Homelab"
whitelist:
  reason: "Permitir solo Windows y Localhost; dejar a Kali fuera"
  ip:
    - "127.0.0.1"
    - ":::1"
    - "192.168.0.32" # <--- AQUÍ SOLO LA IP DE NUESTRO WINDOWS (Sin
el /24)
# Hemos quitado las líneas de 'cidr' para que el resto de la red NO sea ignorada.
```

```
GNU nano 7.2 /etc/crowdsec/parsers/s02-enrich/whitelists.yaml
name: crowdsecurity/whitelists
description: "Whitelist selectiva para Homelab"
whitelist:
  reason: "Permitir solo Windows y Localhost; dejar a Kali fuera"
  ip:
    - "127.0.0.1"
    - "::1"
    - "192.168.0.32"
```

¿Por qué lo hacemos así?

1. **Por qué quitar 192.168.0.0/16**

Si dejamos esa línea, CrowdSec ignorará cualquier ataque que venga de 192.168.x.x. Nuestro Kali vive ahí, así que CrowdSec nunca lo vería como una amenaza. Al quitarlo, CrowdSec empieza a "vigilar" la red local.

2. **Por qué usar ip: y no cidr: para nuestro equipo anfitrión Windows**

* **192.168.0.32 (con ip:) le dice:** *"Confía solo en esta máquina específica".*

- **192.168.0.32/24 (con cidr:) es un error común:** el /24 le dice que confíe en toda la red del .0 al .255.
- **Regla de oro:** Para una sola máquina, usar la IP limpia.

Reiniciamos:

```
sudo systemctl restart crowdsec
```

15.2 Detectar escaneos comunes de Nmap y otros

```
sudo cscli collections install crowdsecurity/iptables
```

```
ozone@ubuntuuserver:~$ sudo cscli collections install crowdsecurity/iptables
Action plan:
📦 download
collections: crowdsecurity/iptables (0.3)
contexts: crowdsecurity/firewall_base (0.2)
scenarios: crowdsecurity/iptables-scan-multi_ports (0.3)
parsers: crowdsecurity/iptables-logs (0.7)
✅ enable
collections: crowdsecurity/iptables
contexts: crowdsecurity/firewall_base
scenarios: crowdsecurity/iptables-scan-multi_ports
parsers: crowdsecurity/iptables-logs

downloading parsers:crowdsecurity/iptables-logs
downloading scenarios:crowdsecurity/iptables-scan-multi_ports
downloading contexts:crowdsecurity/firewall_base
downloading collections:crowdsecurity/iptables
enabling parsers:crowdsecurity/iptables-logs
enabling scenarios:crowdsecurity/iptables-scan-multi_ports
enabling contexts:crowdsecurity/firewall_base
enabling collections:crowdsecurity/iptables

Run 'sudo systemctl reload crowdsec' for the new configuration to be effective.
ozone@ubuntuuserver:~$ |
```

Asegurarse de que quede así al final (usar espacios, no tabuladores):

```
GNU nano 7.2
filenames:
- /var/log/ufw.log
labels:
type: ufw|
```

Reiniciamos y verificamos

```
sudo systemctl restart crowdsec
sudo cscli metrics
```


(Ahora en *metrics* veremos qué */var/log/ufw.log* tiene "Lines read").

```
ozone@ubuntuuserver:~$ sudo cscli metrics
```

Acquisition Metrics			
Source whitelisted	Lines read	Lines parsed	Lines unparsed
file:/var/log/auth.log	3	-	3

Una vez que veamos en *metrics* que las líneas de */var/log/ufw.log* se están leyendo, vamos a nuestro **Kali** y lanzamos el escaneo agresivo de nuevo:

```
nmap -sV -sC -p- -T4 192.168.0.35
```

En **Ubuntu**, ejecuta: `sudo cscli decisions list`

Veremos la IP de Kali (192.168.0.38) con una decisión de **ban** activa.

```
ozone@ubuntuuserver:~$ sudo cscli decisions list
```

ID	Source	Scope:Value	Reason	Action	Country
Events	expiration	Alert ID			
4860116	crowdsec 3h59m38s	Ip:192.168.0.38 8	crowdsecurity/iptables-scan-multi_ports	ban	

15.3 Desbloquear IPs bloqueadas.

Si por error nos bloqueamos a nosotros mismos o necesitamos desbloquear por ejemplo la ip de Kali, podemos desbloquearnos desde la consola del servidor Ubuntu con:

```
sudo cscli decisions delete --ip [TU_IP_BLOQUEADA]
```

```
ozone@ubuntuserver:~$ sudo cscli decisions delete --ip 192.168.0.38  
INFO 1 decision(s) deleted  
ozone@ubuntuserver:~$
```

Si revisamos de nuevo

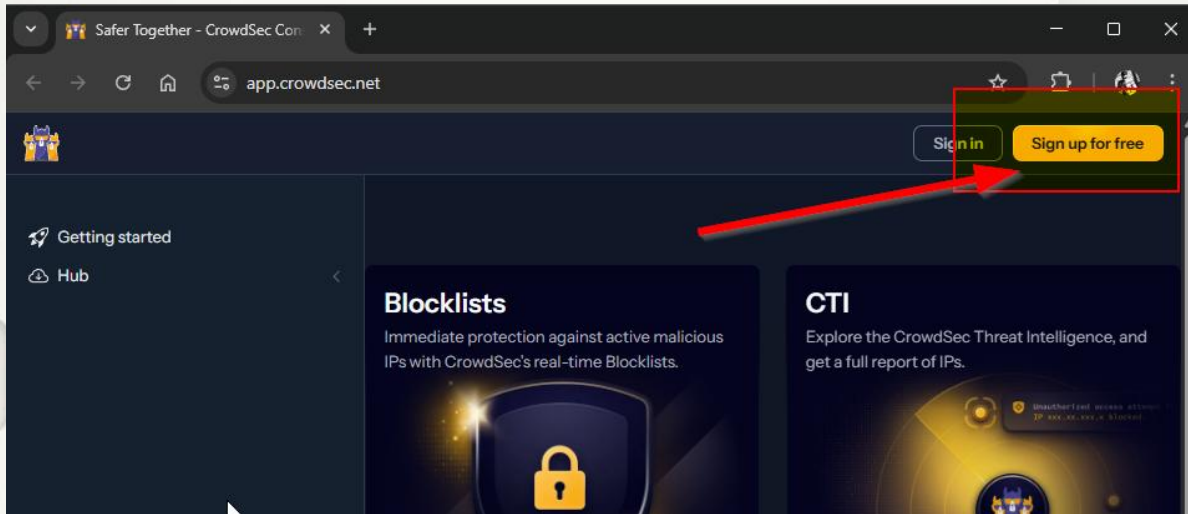
```
ozone@ubuntuserver:~$ sudo cscli decisions list  
No active decisions  
ozone@ubuntuserver:~$
```

Para borrar TODOS los baneos activos (limpieza total)

```
sudo cscli decisions delete --all
```

15.4 Activar el Dashboard visual

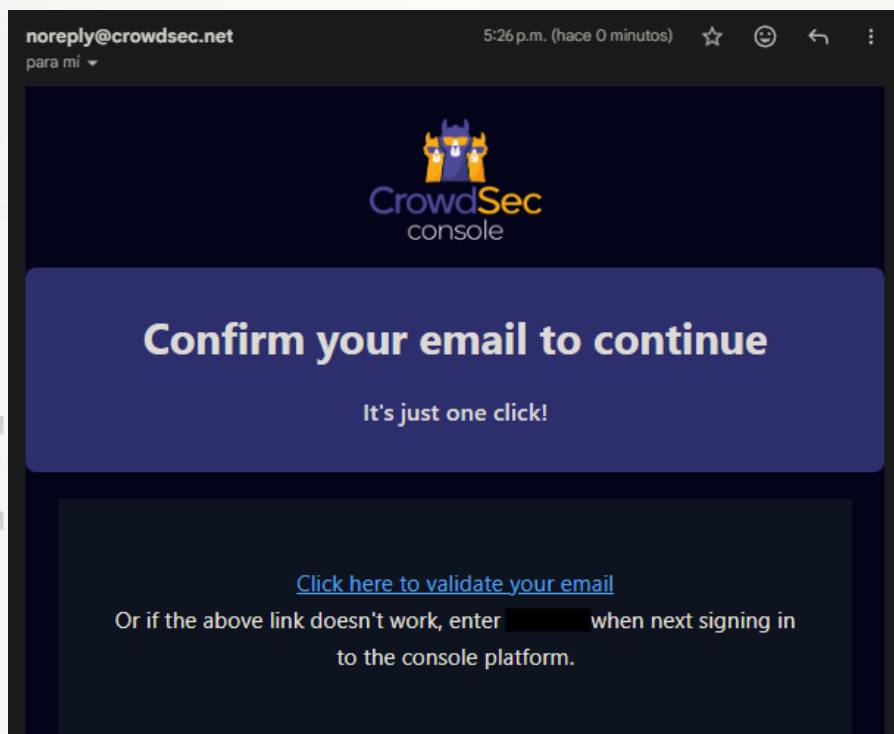
Ir al sitio oficial de crowdsec <https://www.crowdsec.net/>



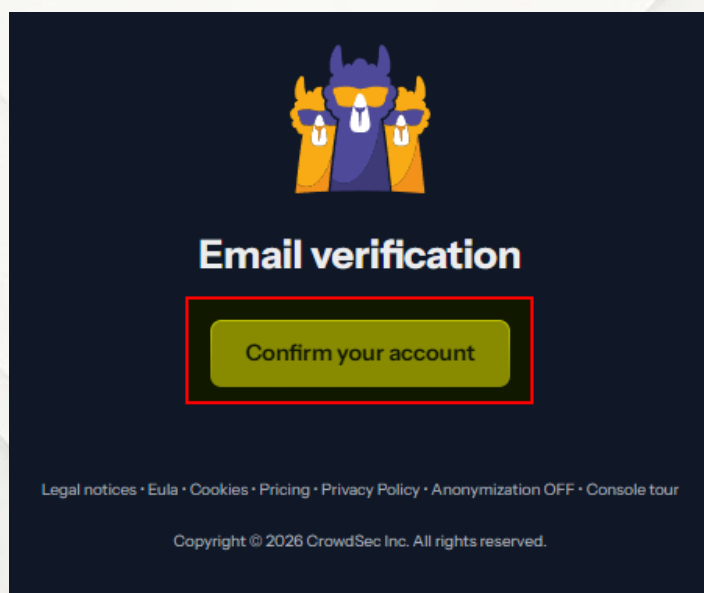
Nos creamos una cuenta con un correo valido.

A screenshot of the 'Create your account' form on the CrowdSec website. The form is titled 'Create your account' and includes the instruction 'Please fill in the details to get started.' It offers two options for account creation: using a Google account or a GitHub account. Below these, there is a section for manual account creation with fields for 'Email address' and 'Password'. The 'Email address' field contains a placeholder ending in '@mail.com' and is highlighted with a red box. The 'Password' field is also highlighted with a red box. Below the password field, there is a checkbox with a checkmark and the text 'I agree to the end user license and I acknowledge that I have read the privacy policy.' At the bottom of the form, there is a 'Sign up' button highlighted with a red box. A link for 'Sign in' is provided for users who already have an account.

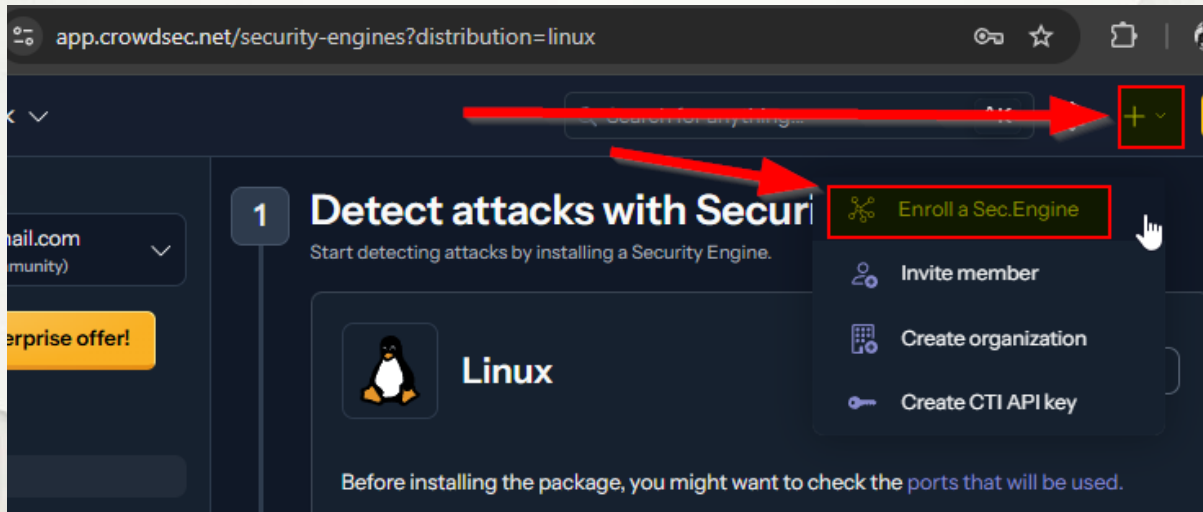
Nos llegará un correo de confirmación de la nueva cuenta:



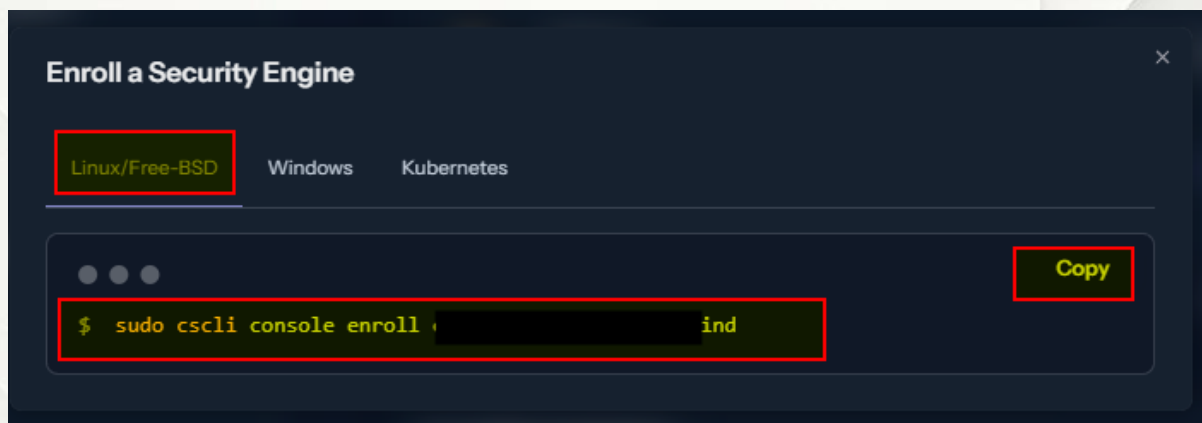
Nos redirigirá a la página y le damos a confirmar cuenta:



Una vez dentro iremos a la derecha y en el signo “+” seleccionaremos “Enroll...”



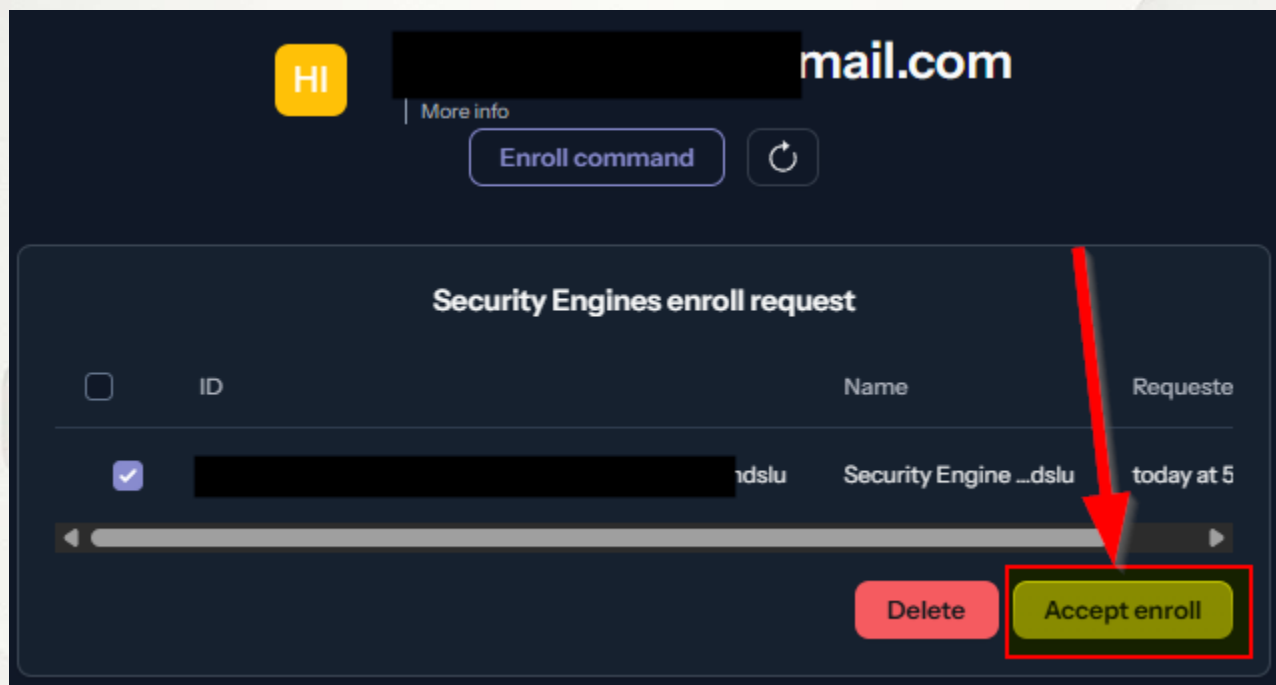
En este caso es para nuestro SO Linux, hacemos click en COPY para copiar el comando:



Pegamos el comando en nuestra consola y ejecutamos, como nos indica debemos reiniciar crowdsec con `sudo systemctl restart crowdsec`

```
ozone@ubuntuserver:~$ sudo cscli console enroll [redacted] ind
[sudo] password for ozone:
INFO manual set to true
INFO context set to true
INFO Enabled manual : Forward manual decisions to the console
INFO Enabled tainted : Forward alerts from tainted scenarios to the console
INFO Enabled context : Forward context with alerts to the console
INFO Watcher successfully enrolled. Visit https://app.crowdsec.net to accept it.
INFO Please restart crowdsec after accepting the enrollment.
ozone@ubuntuserver:~$
```

Desde nuestra maquina anfitriona, volvemos a la página (la que no hemos cerrado) y recibiremos una notificación que nos indica que debemos aceptar el enrolamiento.



Listo. Si confirmamos el status en consola del enrolamiento debería aparece Activado.

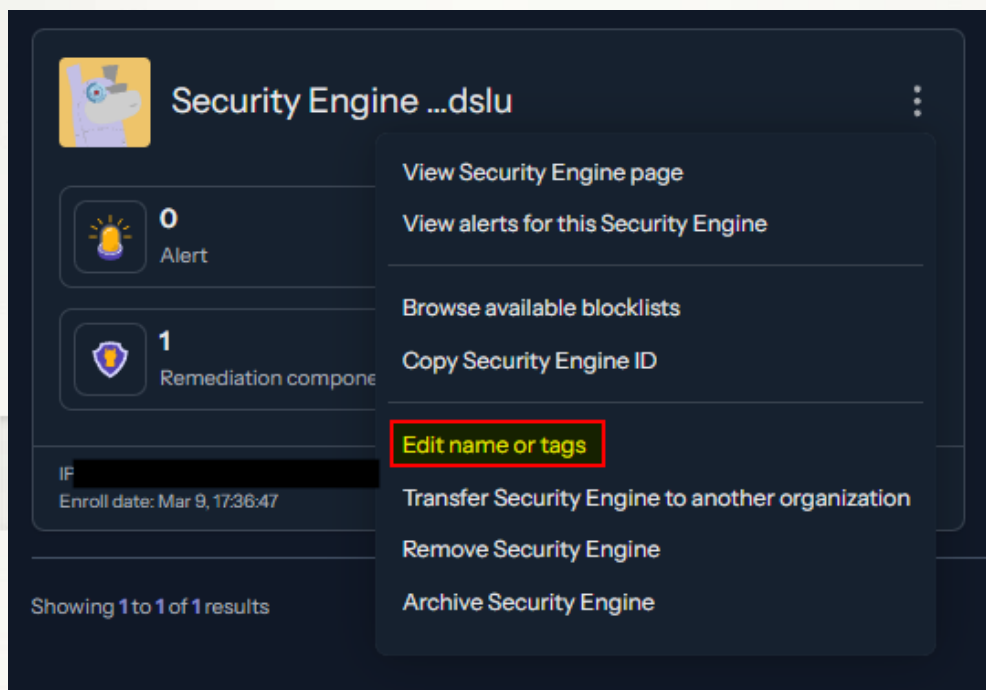
```
sudo cscli console status
```

```
ozone@ubuntu-server:~$ sudo cscli console status
```

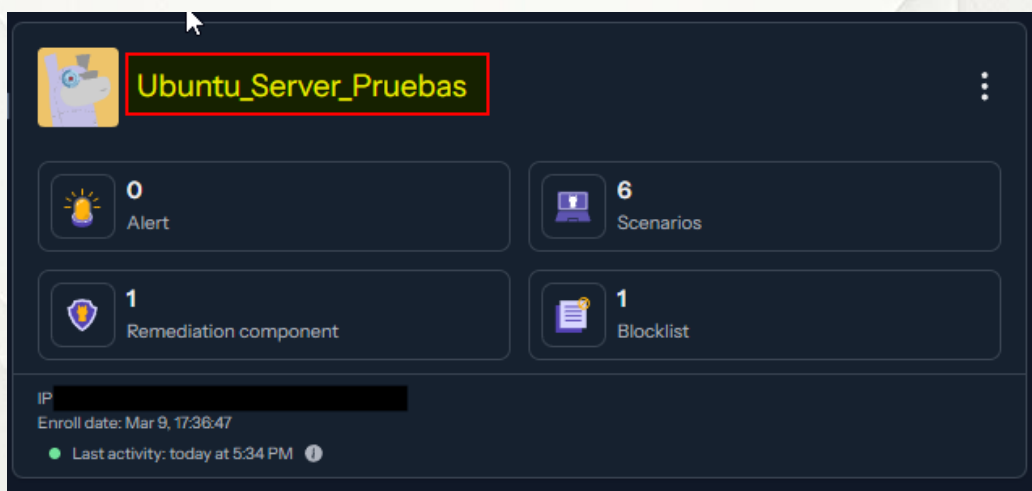
Option Name	Activated	Description
custom	✓	Forward alerts from custom scenarios to the console
manual	✓	Forward manual decisions to the console
tainted	✓	Forward alerts from tainted scenarios to the console
context	✓	Forward context with alerts to the console
console_management	✗	Receive decisions from console

```
ozone@ubuntu-server:~$ |
```

Es una buena práctica cambiar el nombre, sobre todo en entornos con varios servidores; lo podemos hacer desde la web desde los 3 puntos e ir a “Edit name...”.



Y ya tenemos nuestro servidor en Crowdsec con nuevo nombre.



Listo!!! Con esto ya tenemos el motor instalado, el "escudo" (bouncer) conectado al firewall, nuestra IP de Windows protegida en la lista blanca y la conexión visual con el Dashboard.

16. Logs: Del análisis manual al automatizado.

Si bien en una guía anterior ya habíamos hablado de los logs, es bueno complementar esta información ahora que tenemos nuevas herramientas en nuestro servidor.

16.1. Método tradicional

Como practica de seguridad es recomendable revisar regularmente:

`/var/log/auth.log`

`/var/log/syslog`

`/var/log/ufw.log`

`/var/log/fail2ban.log`

Si hay web:

`/var/log/nginx/access.log`

Si bien revisar los archivos .log directamente es la "fuente de la verdad", ahora tenemos herramientas que procesan esa información por nosotros.

16.2. Método moderno.

Si nos basamos en la configuración de las herramientas de esta guía podemos decir:

- **Fail2Ban** ya está leyendo `/var/log/auth.log`.
- **CrowdSec** ya está analizando `/var/log/ufw.log` y logs de red.
- **Ubuntu 24.04** utiliza systemd, por lo que muchos logs se consultan mejor con `journalctl`.

16.3. Tabla comparativa entre métodos.

Recurso	Método Tradicional (Lento)	Método Moderno (Recomendado)
Eventos de red	<code>tail -f /var/log/ufw.log</code>	<code>sudo cscli metrics</code> (Muestra qué se está bloqueando en tiempo real)
Accesos SSH	<code>grep "Failed" /var/log/auth.log</code>	<code>sudo journalctl -u ssh -n 20</code> (Últimos 20 eventos del servicio)
Acciones de bloqueo	<code>cat /var/log/fail2ban.log</code>	<code>sudo cscli alerts list</code> (Historial de ataques detectados con contexto)



17. Monitoreo: Comandos "Old School" vs. "Pro"

Aquí es donde realmente brilla nuestra nueva configuración. El uso de grep y awk es una habilidad fundamental, pero en el día a día, los comandos de las herramientas que instalamos nos dan datos "ya masticados". Acá vemos algunos ejemplos de cómo dialogan con herramientas tradicionales.

17.1. Monitoreo de red (Método tradicional)

```
sudo ss -tulpn
```

Sigue siendo el estándar de oro. Nos dice qué puertas están abiertas en este preciso instante. Debemos usarlo siempre para verificar que no haya servicios "colados".

17.2. Detección de ataques (Método tradicional)

- Detectar intentos SSH.

```
grep "Failed password" /var/log/auth.log
```

- Ver IP atacando.

```
grep "Failed password" /var/log/auth.log | awk '{print $11}' |  
sort | uniq -c
```

17.2.1 Detección de ataques (Método moderno)

En lugar de procesar texto plano con grep, utilizamos la potencia de los motores que ya están corriendo:

- **Ver el "Top" de atacantes (CrowdSec):**

```
sudo cscli decisions list
```

¿Por qué es mejor?

Nos dice la IP, el motivo, el país de origen y cuánto tiempo le queda de baneo.

- **Ver efectividad de las reglas (Fail2Ban):**

```
sudo fail2ban-client status
```

¿Por qué es mejor?

Nos da una visión de pájaro de qué "jaula" (ssh, portscan, recidive) está trabajando más.

- **Análisis forense rápido (CrowdSec):**

```
sudo cscli alerts inspect [ID_DEL_ALERTA]
```

¿Por qué es mejor?

Nos muestra la línea exacta del log que activó la alerta sin que tengamos que buscarla manualmente.

18. Cambiar puerto SSH.

Esto por ahora es opcional. Yo no lo haré pero es una buena práctica de seguridad en servidores en producción.

Archivo:

```
sudo nano /etc/ssh/sshd_config
```

Ejemplo.

Port 2222

Si lo cambias, recuerda antes de cerrar el puerto 22:

```
sudo ufw allow [tu-nuevo-puerto]/tcp
```

19. Desactivar root.

Esto ya lo habíamos hecho en el Laboratorio 3 pero no está de más recordarlo.

Editamos el archivo:

```
sudo nano /etc/ssh/sshd_config
```

y confirmamos que tenemos activado lo siguiente

PermitRootLogin no

```
# Authentication:
```

```
LoginGraceTime 30
```

```
PermitRootLogin no
```

```
#StrictModes yes
```

```
MaxAuthTries 3
```

```
#MaxSessions 10
```


20. Usar autenticación por llaves.

Para esto es muy importante el orden con el objetivo de evitar el temido “lockout”.

Lo primero que haremos inicialmente será apagar nuestra VM de Ubuntu y crearemos un snapshot o punto de restauración de la máquina virtual, muy importante por si algo no sale bien.

A continuación abriremos una conexión por SSH desde nuestra maquina local (en mi caso Windows) al servidor Ubuntu, ingresaremos como siempre con nuestra contraseña y la dejaremos ahí.

Desde Powershell o VSCode (o el hipervisor que estes utilizando, yo estoy desde Powershell) nos conectamos por SSH al servidor Ubuntu.

```
ssh tu_usuario_ubuntu@192.168.x.x
```

20.1. Generar las llaves en nuestra máquina local (Windows)

Abrir una terminal (PowerShell o CMD) en nuestro Windows y escribimos:

```
ssh-keygen -t ed25519
```

(Le damos Enter a todo. Yo no haré lo siguiente en esta ocasión, pero si quieres máxima seguridad, ponle una "passphrase"). Esto creará dos archivos en **C:\Users\TuUsuario\ssh**. El archivo **.pub** es el que enviaremos al servidor.

```

PS C:\Users\ruben> ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\ruben/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\ruben/.ssh/id_ed25519
Your public key has been saved in C:\Users\ruben/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256: 61U @DESKTOP-8UL
The key's randomart image is:
+--[ED25519 256]--+
|o
|.
|o
|.
+----[SHA256]-----+
PS C:\Users\ruben>

```

20.2. Copiar la llave al servidor Ubuntu

En la misma terminal ingresamos el siguiente comando para copiar la llave. (Reemplaza con tus credenciales).

```
ssh-copy-id -i $HOME\.ssh\id_ed25519.pub tu_usuario_ubuntu@192.168.x.x
```

Si te aparece el siguiente error no te preocupes.

```

PS C:\Users\ruben> ssh-copy-id -i $HOME\.ssh\id_ed25519.pub ozone@192.168.0.36
ssh-copy-id : El término 'ssh-copy-id' no se reconoce como nombre de un cmdlet, función, archivo de script o programa
ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta
es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ ssh-copy-id -i $HOME\.ssh\id_ed25519.pub ozone@192.168.0.36
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (ssh-copy-id:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

```

El comando `ssh-copy-id` no viene instalado por defecto en Windows (a diferencia de Linux/Mac).

Si al ejecutarlo (como a mi) te da este error de *"comando no se reconoce"*, puedes hacerlo de forma manual con este "truco" de PowerShell que hace exactamente lo mismo (*Reemplaza con tus credenciales*):

```
type $HOME\.ssh\id_ed25519.pub | ssh tu_usuario_ubuntu @192.168.x.x  
"mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"
```

```
PS C:\Users\ruben> type $HOME\.ssh\id_ed25519.pub | ssh ozone@192.168.0.36 "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_  
keys"  
Ubuntu 24.04.4 LTS  
Acceso restringido.  
Toda actividad es monitoreada.  
ozone@192.168.0.36's password:  
PS C:\Users\ruben> |
```



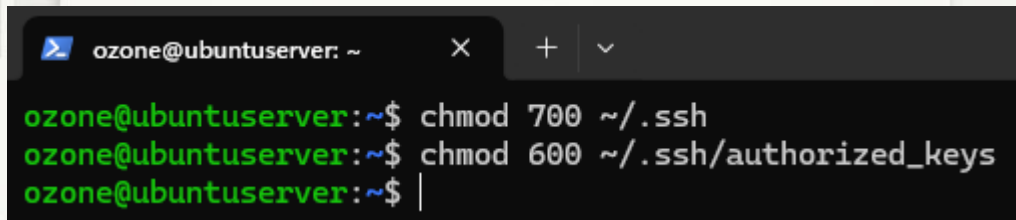
20.3. Permisos en el servidor

Una vez que copiemos la llave, debemos asegurarnos de que los permisos en Ubuntu sean estrictos (SSH es muy "especial" con esto y si los permisos son muy abiertos, ignorará la llave por seguridad):

En nuestra terminal de Ubuntu que dejamos abierta al principio ejecutamos:

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/authorized_keys
```

A screenshot of a terminal window titled 'ozone@ubuntu-server: ~'. The terminal shows three lines of commands being executed: 'ozone@ubuntu-server:~\$ chmod 700 ~/.ssh', 'ozone@ubuntu-server:~\$ chmod 600 ~/.ssh/authorized_keys', and 'ozone@ubuntu-server:~\$ |'. The prompt and command text are green, while the cursor is a white vertical bar.

```
ozone@ubuntu-server: ~  
ozone@ubuntu-server:~$ chmod 700 ~/.ssh  
ozone@ubuntu-server:~$ chmod 600 ~/.ssh/authorized_keys  
ozone@ubuntu-server:~$ |
```

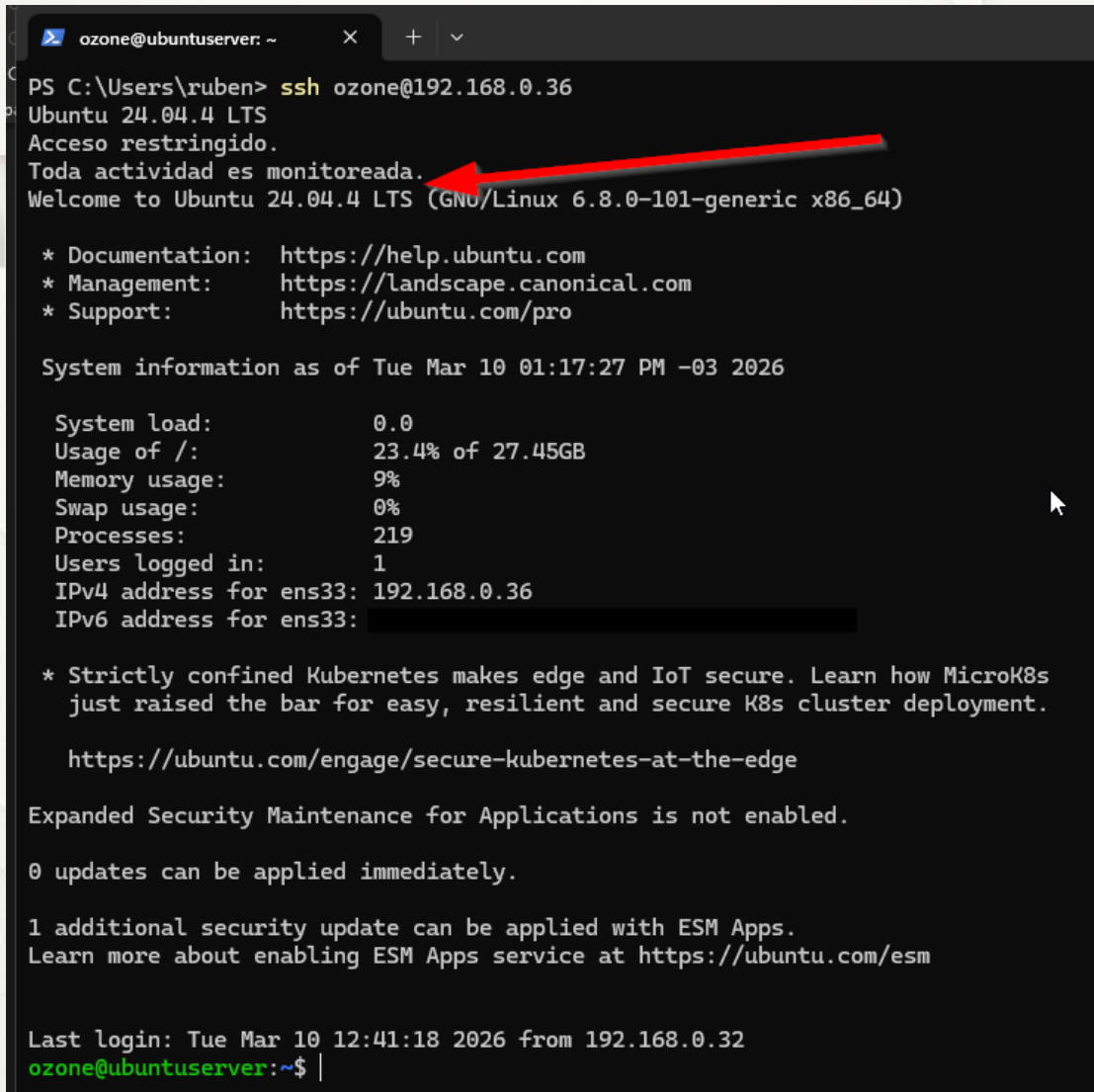

20.4. Prueba de oro

Abriremos una nueva terminal en Windows e intentamos entrar:

```
ssh ozone@192.168.0.36
```

Si NO pide contraseña: Todo está OK.

Si pide contraseña: STOP, la llave no está funcionando aún y hay que ejecutar el proceso de nuevo.



```
PS C:\Users\ruben> ssh ozone@192.168.0.36
Ubuntu 24.04.4 LTS
Acceso restringido.
Toda actividad es monitoreada.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Mar 10 01:17:27 PM -03 2026

System load:          0.0
Usage of /:            23.4% of 27.45GB
Memory usage:         9%
Swap usage:           0%
Processes:            219
Users logged in:      1
IPv4 address for ens33: 192.168.0.36
IPv6 address for ens33:

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Tue Mar 10 12:41:18 2026 from 192.168.0.32
ozone@ubuntuserver:~$ |
```

En la imagen se puede apreciar cómo donde antes me pedía contraseña ya puedo acceder con la llave criptográfica creada.

20.5 Deshabilitar contraseñas

Estas líneas las habíamos editado en el laboratorio anterior y en ese caso se dejó claro que las cambiaríamos cuando llegáramos hasta este punto.

Desde nuestra primera terminal de Ubuntu, la que dejamos abierta al principio, editamos el archivo:

```
sudo nano /etc/ssh/sshd_config
```

y editamos estas líneas:

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

```
KbdInteractiveAuthentication no
```

```
LoggingGraceTime 30
PermitRootLogin no
#StrictModes yes
MaxAuthTries 3
#MaxSessions 10
PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication no
PermitEmptyPasswords no

# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
KbdInteractiveAuthentication no
# Kerberos options
```

Guarda y salimos (Ctrl+O, Enter, Ctrl+X).

Reinicia el servicio: `sudo systemctl restart ssh`

20.6 Si no resulta

Abrimos una nueva terminal y nos conectamos por SSH, debería dejarnos entrar directo sin contraseña. Si es así, ya puedes cerrar todas las otras terminales y tendríamos listo el acceso por llaves que es mucho más seguro.

Si no hubiese resultado, el problema habría estado en uno de estos **3 puntos clave**, y solo habríamos tenido que revisar ese punto:

1. El archivo `authorized_keys` (El más común)

Si la llave no funciona, lo primero es verificar si el contenido se copió bien.

- **Error común:** El comando de copiado pegó la llave como una sola línea interminable o se comió algún carácter al principio (`ssh-ed25519...`).
- **Solución:** Entrar al servidor y hacer un `cat ~/.ssh/authorized_keys` para ver que la llave esté ahí y se vea igual que nuestro archivo `.pub` de Windows.

2. Los permisos (El "muro" invisible)

SSH es extremadamente paranoico. Si los permisos de nuestra carpeta o archivo son demasiado "permisivos" (por ejemplo, si otros usuarios pueden leerlos), el servidor **ignora la llave por seguridad** y nos pide la contraseña por defecto.

- **La regla de oro:** * Carpeta `.ssh`: 700 (Solo nosotros entramos).
 - Archivo `authorized_keys`: 600 (Solo nosotros leemos/escribimos).

3. El archivo de configuración (`sshd_config`)

A veces el servidor tiene la autenticación por llave desactivada de fábrica (aunque en Ubuntu 24 viene activa por defecto).

- **Solución:** Revisar que `PubkeyAuthentication` diga `yes`.

21. Instalar detector de rootkits (RKHunter).

RKHunter (Rootkit Hunter) es un escáner de seguridad que busca "huellas digitales" de software malicioso escondido en lo profundo del sistema. Su trabajo se divide en tres acciones clave:

- **Comparar binarios**

Revisa si comandos básicos (como ls, ps o top) han sido reemplazados por versiones falsas que ocultan procesos atacantes.

- **Buscar rootkits**

Rastrea carpetas y archivos ocultos conocidos por ser usados por troyanos y puertas traseras (backdoors).

- **Auditar el sistema**

Verifica permisos sospechosos y configuraciones de red peligrosas.

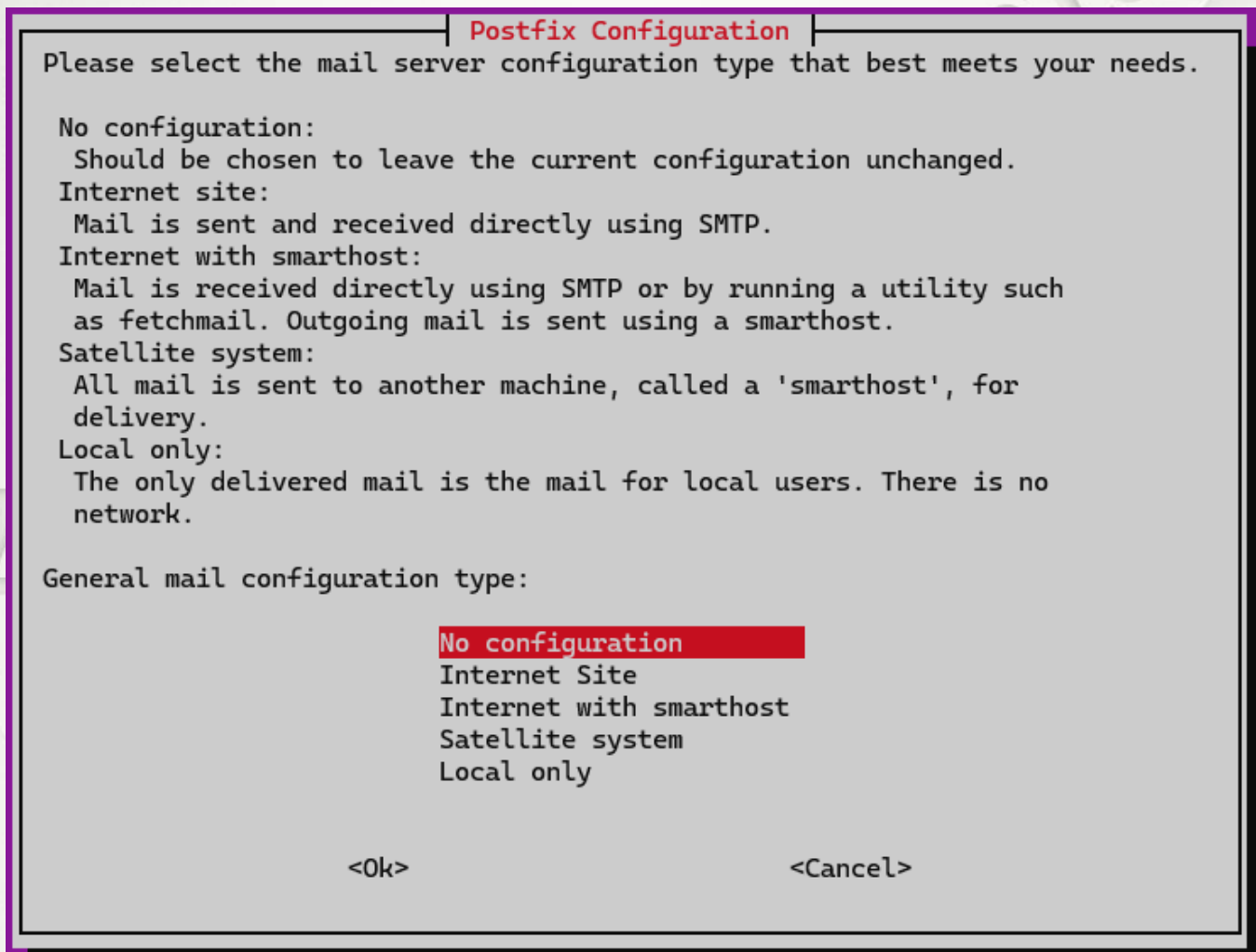
Es, básicamente, el perro guardián que olfatea si alguien logró entrar y "plantar" algo para quedarse con el control de tu servidor.

21.1 Instalación

```
sudo apt update && sudo apt install rkhunter -y
```

Si aparece el cuadro que muestra la imagen siguiente, es porque rkhunter tiene la capacidad de enviar reportes por correo electrónico, y para ello intenta configurar **Postfix** (un servidor de correo).

Como este es un servidor de laboratorio y probablemente no necesitemos que el servidor envíe correos al mundo exterior directamente (lo cual es complejo de configurar), elegiremos "No configuration".



¿Esto quiere decir que si configuro alguna aplicación que envíe correos ... no podrá?

Exactamente. Si elegimos *"No configuration"*, el servidor no tendrá instalado el "motor" necesario para sacar correos hacia el mundo exterior (Gmail, Outlook, etc.).

Sin embargo, no hay que preocuparse, esto es lo correcto por ahora y aquí explico por qué:

1. El riesgo de un servidor de correo abierto

Instalar un servidor de correo (Postfix) y abrirlo al internet sin una configuración **experta** es peligroso. Los bots lo detectarán en minutos y tratarán de usar nuestra IP para enviar miles de

correos de SPAM, lo que provocará que la IP de nuestro servidor (192.168.0.36 o nuestra IP pública) caiga en listas negras globales.

2. ¿Qué pasa si necesitamos enviar correos después?

Si en el futuro instalamos una aplicación que deba enviar notificaciones (por ejemplo, una alerta de seguridad o un formulario web), tenemos dos caminos mejores que configurar Postfix ahora:

- **SMTP Externo (Recomendado):** Configuramos nuestra aplicación para que use una cuenta de Gmail, SendGrid o Outlook (igual que configuramos Outlook en nuestro PC). No necesitamos Postfix en nuestro servidor para esto.
- **Reconfigurar Postfix:** Si llegamos a necesitarlo obligatoriamente, siempre podemos volver a ese menú con el comando: `sudo dpkg-reconfigure postfix`

3. El enfoque del Blue Team

En seguridad, aplicamos el principio de *"Lo que no se usa, no se instala"*. Como nuestro objetivo actual es endurecer el servidor (Hardening), tener un servidor de correo inactivo reduce la superficie de ataque.

21.2. Configuración de la "Línea Base" (Baseline)

Este es el paso más importante. **rkhunter** funciona comparando tus archivos actuales contra una base de datos. Debes decirle que confíe en los archivos actuales (que acabas de asegurar):

21.2.1. Actualizamos la base de datos interna de rkhunter

```
sudo rkhunter --update
```

me lanza el siguiente error.

```
ozone@ubuntu-server:~$ sudo rkhunter --update
[sudo] password for ozone:
Invalid WEB_CMD configuration option: Relative pathname: "/bin/false"
ozone@ubuntu-server:~$
```

Lo primero es señalar que ese error es muy común en Ubuntu 24.04. Sucede porque, por defecto, el archivo de configuración de rkhunter apunta a `/bin/false` para las actualizaciones web, pero lo hace de una forma que el programa considera "inválida" o insegura.

Como seleccionamos "No configuration" en Postfix y estamos en un entorno controlado, vamos a corregir esto rápidamente para que podamos hacer el update.

Solución:

Editar el archivo de configuración principal de rkhunter para corregir la ruta de los comandos.

```
sudo nano /etc/rkhunter.conf
```

Buscamos la línea de WEB_CMD:

Presionar Ctrl + W y escribir WEB_CMD para encontrarlo más rápido. Encontrarás algo como esto: **WEB_CMD="/bin/false"**

```
#
#   WEB_CMD="ftp -o -"
#
# This option has no default value.
#
WEB_CMD="/bin/false"

#
# Set the following option to '1' if locking is to be used when rkhunter runs.
# The lock is set just before logging starts, and is removed when the program
# ends. It is used to prevent items such as the log file, and the file
```

Cambiarlo por una ruta absoluta válida o coméntalo: Si no vamos a actualizar rkhunter vía web (lo cual es normal, ya que se actualiza con apt), simplemente nos aseguramos de que use el comando **curl** o **wget** que sí están en nuestro sistema.

Buscamos también la línea **UPDATE_MIRRORS** y la dejamos en 1 si queremos que intente actualizarse ya que tenemos salida por la conexión BRIDGE de la VM.

Lo anterior llevado a la práctica:

Buscar estas 3 líneas y las dejamos así (quitando el # si lo tienen):

```
UPDATE_MIRRORS=1
MIRRORS_MODE=0
WEB_CMD=""
```



```
#
# The default value is '1'.
#
UPDATE_MIRRORS=1

#
# The MIRRORS_MODE option tells rkh
# the '--update' or '--versioncheck
# Possible values are:
#   0 - use any mirror
#   1 - only use local mirrors
#   2 - only use remote mirrors
#
# Local and remote mirrors can be d
# 'local=' and 'remote=' keywords r
#
# The default value is '0'.
#
MIRRORS_MODE=0

#
# Email a message to this address i
# being checked. Multiple addresses
```

```
#   WEB_CMD="ftp -o -"
#
# This option has no default value.
#
WEB_CMD=""
#
```

Al dejar WEB_CMD vacío, rkhunter intentará usar automáticamente herramientas estándar como wget o curl que ya tenemos en /usr/bin/.

Guardar y salir.

Lanzamos nuevamente el comando para actualizar:

```
sudo rkhunter --update
```

```
ozone@ubuntuerver:~$ sudo rkhunter --update
[sudo] password for ozone:
[ Rootkit Hunter version 1.4.6 ]

Checking rkhunter data files...
  Checking file mirrors.dat           [ Updated ]
  Checking file programs_bad.dat      [ No update ]
  Checking file backdoorports.dat     [ No update ]
  Checking file suspscan.dat          [ No update ]
  Checking file i18n/cn               [ Skipped ]
  Checking file i18n/de               [ Skipped ]
  Checking file i18n/en               [ No update ]
  Checking file i18n/tr               [ Skipped ]
  Checking file i18n/tr.utf8          [ Skipped ]
  Checking file i18n/zh               [ Skipped ]
  Checking file i18n/zh.utf8          [ Skipped ]
  Checking file i18n/ja               [ Skipped ]
ozone@ubuntuerver:~$ |
```

21.2.2 Captura las propiedades de nuestros archivos actuales (el "estado sano")

Este comando "saca la foto" de nuestros archivos actuales. Sin este paso, el siguiente escaneo nos dará cientos de alertas rojas porque rkhunter no sabrá qué archivos son los nuestros y cuáles son sospechosos.

```
sudo rkhunter --propupd
```

Nota: Cada vez que actualicemos nuestro servidor con `apt upgrade`, es buena práctica volver a ejecutar `sudo rkhunter --propupd` para que no detecte las actualizaciones legítimas como ataques.

```
ozone@ubuntuuserver:~$ sudo rkhunter --propupd
[ Rootkit Hunter version 1.4.6 ]
File updated: searched for 180 files, found 142
```

Esa es una salida correcta. Lo que acaba de ocurrir es que rkhunter recorrió las rutas más críticas de mi sistema (como `/bin`, `/sbin`, `/usr/bin`) y "tomó una fotografía" (un hash criptográfico) de cada archivo importante que encontró.

¿Qué significa ese resultado?

- **Searched for 180 files:** Buscó 180 archivos que suelen ser blanco de ataques (como `ls`, `ps`, `login`, `ssh`).
- **Found 142:** Encontró 142 de ellos instalados en mi Ubuntu Server 24. Los otros 38 no están presentes (lo cual es normal, ya que algunas herramientas varían según la versión de Linux).
- **File updated:** Guardó la firma de esos 142 archivos en su base de datos local (`/var/lib/rkhunter/db/rkhunter.dat`).

21.2.3 “El gran escaneo”

Ahora que el "perro guardián" ya conoce el olor de nuestros archivos sanos, vamos a soltarlo para que busque amenazas reales. Ejecutamos:

```
sudo rkhunter --check --sk
```

(El --sk es para que no se detenga a pedirnos que presionemos Enter en cada sección).

Capturas de ejemplo

```
ozone@ubuntu:~$ sudo rkhunter --check --sk
[ Rootkit Hunter version 1.4.6 ]

Checking system commands...

Performing 'strings' command checks
Checking 'strings' command [ OK ]

Performing 'shared libraries' checks
Checking for preloading variables [ None found ]
Checking for preloaded libraries [ None found ]
Checking LD_LIBRARY_PATH variable [ Not found ]

Performing file properties checks
Checking for prerequisites [ OK ]
/usr/sbin/adduser [ OK ]
/usr/sbin/chroot [ OK ]
/usr/sbin/cron [ OK ]
/usr/sbin/depmod [ OK ]
/usr/sbin/fsck [ OK ]
/usr/sbin/groupadd [ OK ]

Checking for rootkits...

Performing check of known rootkit files and directories
55808 Trojan - Variant A [ Not found ]
ADM Worm [ Not found ]
AjaKit Rootkit [ Not found ]
Adore Rootkit [ Not found ]
aPa Kit [ Not found ]
Apache Worm [ Not found ]
Ambient (ark) Rootkit [ Not found ]
Balaor Rootkit [ Not found ]
BeastKit Rootkit [ Not found ]
beX2 Rootkit [ Not found ]
```


Checking the network...

Performing checks on the network ports
Checking for backdoor ports [None found]

Performing checks on the network interfaces
Checking for promiscuous interfaces [None found]

Checking the local host...

Performing system boot checks
Checking for local host name [Found]
Checking for system startup files [Found]
Checking system startup files for malware [None found]

21.2.4 Investigar una flag

En la siguiente imagen aparece un "Warning"

```
Checking for a running system logging daemon [ Found ]  
Checking for a system logging configuration file [ Found ]  
Checking if syslog remote logging is allowed [ Not allowed ]  
  
Performing filesystem checks  
Checking /dev for suspicious file types [ None found ]  
Checking for hidden files and directories [ Warning ]  
  
System checks summary  
=====
```

Ese [**Warning**] en la sección de "hidden files and directories" es el clásico rito de iniciación en rkhunter. No significa que mi Ubuntu tenga un virus; significa que el escáner encontró carpetas o archivos que empiezan con un punto (.) en lugares donde normalmente no deberían estar, o que tienen nombres sospechosos.

En **Ubuntu 24.04**, esto suele ser un **falso positivo**. Vamos a investigar qué es exactamente.

Cómo ver qué causó el Warning

rkhunter guarda el detalle exacto en su archivo de registro. Ejecutaremos este comando para ver solo las alertas:

```
sudo grep "Warning" /var/log/rkhunter.log
```

```
ozone@ubuntuserver:~$ sudo grep "Warning" /var/log/rkhunter.log
[15:10:25] Checking for hidden files and directories [ Warning ]
[15:10:25] Warning: Hidden file found: /etc/.resolv.conf.systemd-resolved.bak: ASCII text
[15:10:25] Warning: Hidden file found: /etc/.updated: ASCII text
```

Como no soy un ultra experto y las herramientas son para utilizarlas, copie la salida y le pedí a una IA que me ayude a entender y esto fue lo que me dijo (*Recuerda que en un entorno de ciberseguridad debes tener mucho cuidado con la información que consultas en los modelos de inteligencia artificial, si es un script, log, salida o lo que sea debemos tener muy claro de no compartir información que comprometa datos sensibles*):

Cito a la ia respecto de estos logs:

“Esos son falsos positivos clásicos de Ubuntu y puedes estar 100% tranquilo. Tu servidor no está infectado.

Aquí te explico qué son esos archivos para que los pongas en tu reporte técnico:

1. /etc/.resolv.conf.systemd-resolved.bak

Es una copia de seguridad (backup) de la configuración de tus DNS. Ubuntu usa systemd-resolved para gestionar a qué servidores preguntar por nombres de dominio (como <https://www.google.com/search?q=google.com>). Al ser un archivo que empieza con un punto (.), rkhunter lo marca porque los atacantes a veces esconden configuraciones de red falsas en archivos ocultos dentro de /etc.

Veredicto: SEGURO. Es un archivo residual del sistema.

2. /etc/.updated

Este archivo es simplemente una "estampa de tiempo" (timestamp). Ubuntu lo crea o lo toca cada vez que se realiza una actualización del sistema o de paquetes para saber cuándo fue la última vez que el sistema estuvo "al día".

Veredicto: SEGURO. *Es un marcador de mantenimiento de paquetes.”*

¿Cómo dejar el reporte en "limpio"? (Opcional)

Si queremos que la próxima vez que corramos `sudo rkhunter --check --sk` no aparezcan esos Warnings en rojo, podemos "darle permiso" a esos archivos específicos:

Editamos la configuración:

```
sudo nano /etc/rkhunter.conf
```

Buscamos la sección de archivos permitidos (también podemos ir al final del archivo) y añadimos estas dos líneas:

```
ALLOWHIDDENFILE="/etc/.resolv.conf.systemd-resolved.bak"  
ALLOWHIDDENFILE="/etc/.updated"
```

```
#  
# Allow the specified hidden file to be whitelisted.  
#  
# This option may be specified more than once, and may use wildcard characters.  
#  
# The default value is the null string.  
#  
#ALLOWHIDDENFILE=/usr/share/man/man1/..1.gz  
#ALLOWHIDDENFILE=/usr/bin/.fipscheck.hmac  
#ALLOWHIDDENFILE=/usr/bin/.ssh.hmac  
#ALLOWHIDDENFILE=/usr/lib/.libfipscheck.so.1.1.0.hmac  
#ALLOWHIDDENFILE=/usr/lib/hmaccalc/sha1hmac.hmac  
#ALLOWHIDDENFILE=/usr/lib/hmaccalc/sha256hmac.hmac  
#ALLOWHIDDENFILE=/usr/sbin/.sshd.hmac  
#ALLOWHIDDENFILE=/usr/share/man/man5/.k5login.5.gz  
#ALLOWHIDDENFILE=/usr/share/man/man5/.k5identity.5.gz  
#ALLOWHIDDENFILE=/etc/.gitignore  
#ALLOWHIDDENFILE=/etc/.bzrignore  
#ALLOWHIDDENFILE=/etc/.etckeeper  
ALLOWHIDDENFILE="/etc/.resolv.conf.systemd-resolved.bak"  
ALLOWHIDDENFILE="/etc/.updated"
```

Guardamos y salimos.

22. Monitoreo centralizado (script de Auditoría)

Este script es una herramienta de **Auditoría Automatizada y Detección de Intrusiones (HIDS)** diseñada específicamente para el entorno Ubuntu Server que hemos configurado. Su función principal es actuar como un **vigilante 24/7** que verifica si la "fortaleza" que construimos en los Laboratorios 3 y 4 sigue intacta.

¿Qué hace concretamente este script?

El script ejecuta un escaneo profundo del sistema dividido en cuatro capas lógicas, generando un **informe técnico** y un **resumen ejecutivo** al finalizar.

1. Auditoría de red (Nivel 1)

- **Vigilancia de servicios:** Verifica que UFW, Fail2Ban y CrowdSec estén corriendo. Si alguno cae, genera una alerta inmediata.
- **Verificación de reglas:** Confirma que la política de *"Denegar todo"* (Default Deny) siga activa en el firewall.
- **Test de resiliencia:** Revisa que la protección contra ataques de inundación (**SYN Flood**) esté activa en el núcleo del sistema (sysctl).
- **Prevención de DoS:** Monitorea el espacio en disco (LVM) para evitar que un llenado masivo de logs bloquee el servidor.

2. Control de identidad (Nivel 2)

- **Validación SSH:** Escanea el archivo `sshd_config` para asegurar que nadie haya rehabilitado las contraseñas o el acceso a root por error.
- **Cumplimiento de llaves:** Verifica que los usuarios solo utilicen llaves **ED25519** (el estándar más alto). Si alguien usa RSA o llaves antiguas, el script lo reporta.
- **Higiene de permisos:** Revisa que las carpetas `.ssh` de los usuarios tengan los permisos restrictivos (700) correctos para evitar robos de identidad.

3. Análisis de comportamiento (Nivel 3)

- **Estado de bloqueos:** Reporta cuántas IPs han sido baneadas localmente por **Fail2Ban** y extrae las últimas decisiones de la red global de **CrowdSec**.
- **Salud del hub:** Avisa si los escenarios de inteligencia de amenazas de CrowdSec necesitan actualizarse.
- **Detección de fuerza bruta:** Analiza los logs de las últimas 24 horas y entrega un "Top 5" de las IPs atacantes más persistentes.

4. Integridad del host (Nivel 4)

- **Caza de rootkits:** Ejecuta **RKHunter** de forma inteligente. Si ya corrió ese día, solo lee los resultados para ahorrar recursos; si es la primera vez, hace un escaneo profundo.
- **Auditoría forense:** Utiliza **lsattr** (*list attributes*) para verificar si los binarios críticos (como `sudo` o `sshd`) mantienen el bit **immutable**. Si un binario fue modificado y perdió este bit, lanza una alerta de alta prioridad.
- **Búsqueda de backdoors:** Lista todos los usuarios con shell activa para detectar si un atacante creó un usuario oculto para mantener el acceso.

Sistema de alertas

Finalmente, el script suma todos los hallazgos críticos. Si encuentra algo sospechoso, no solo lo guarda en un log local (`/var/log/blueteam`), sino que puede enviar una notificación inmediata vía **Email** o **Webhook** (Slack/Discord), permitiéndote reaccionar a un ataque antes de que sea demasiado tarde.

```

#!/bin/bash
# =====
# REPORTE DIARIO DE SEGURIDAD - BLUE TEAM UBUNTU
# Versión 2.1 - Defensa en Profundidad (4 niveles)
# =====

set -uo pipefail

# -----
# CONFIGURACIÓN
# -----
RED='\033[0;31m'
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
CYAN='\033[0;36m'
NC='\033[0m'

LOG_DIR="/var/log/blueteam"
LOG_FILE="${LOG_DIR}/auditoria_$(date +%F).log"
ALERTAS=0          # contador de hallazgos críticos
ALERTAS_MSG=()     # mensajes acumulados para el resumen final

# Webhook opcional (Slack/Discord/ntfy). Dejar vacío para deshabilitar.
WEBHOOK_URL=""
# Email opcional (requiere mailutils instalado). Dejar vacío para deshabilitar.
ALERT_EMAIL=""

# -----
# UTILIDADES
# -----
mkdir -p "$LOG_DIR"

log() {
    echo -e "$@" | tee -a "$LOG_FILE"
}

alerta() {
    local msg="$1"
    ALERTAS=$((ALERTAS + 1))
    ALERTAS_MSG+=("$msg")
    log "${RED}  [!] ALERTA: ${msg}${NC}"
}

# Valida que un string sea solo una IP v4
validar_ip() {
    local ip="$1"
    [[ "$ip" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]] || return 1
    IFS='.' read -ra octetos <<< "$ip"
    for o in "${octetos[@]}; do
        (( o >= 0 && o <= 255 )) || return 1
    done
}

# Valida nombre de jail: solo alfanuméricos, guiones y guiones bajos

```

```
validar_jail() {  
    [[ "$1" =~ ^[a-zA-Z0-9_-]+$ ]]  
}  
  
# Envía alerta por webhook (JSON simple compatible con Slack/Discord/ntfy)  
enviar_webhook() {  
    [[ -z "$WEBHOOK_URL" ]] && return  
    local payload  
    payload=$(printf '{"text":"[BlueTeam] %s alertas en %s\\n%s"}' \\  
        "$ALERTAS" "$(hostname)" "$(printf '%s\\n' "${ALERTAS_MSG[@]}")")  
    curl -s -X POST -H 'Content-Type: application/json' \\  
        -d "$payload" "$WEBHOOK_URL" >/dev/null 2>&1 || true  
}  
  
# Envía alerta por email  
enviar_email() {  
    [[ -z "$ALERT_EMAIL" ]] && return  
    command -v mail >/dev/null 2>&1 || return  
    printf '%s\\n' "${ALERTAS_MSG[@]}" | \\  
        mail -s "[BlueTeam] $ALERTAS alertas en $(hostname)" "$ALERT_EMAIL"  
    2>/dev/null || true  
}  
  
# _____  
# VALIDAR PRIVILEGIOS  
# _____  
if ! sudo -n true 2>/dev/null; then  
    echo -e "${RED}[ERROR] Este script requiere privilegios sudo sin contraseña  
(NOPASSWD).${NC}"  
    echo -e "${YELLOW}          Ejecuta: sudo visudo y agrega tu usuario con  
NOPASSWD.${NC}"  
    exit 1  
fi  
  
log "${BLUE}===== ${NC}"  
log "${YELLOW} REPORTE DIARIO DE SEGURIDAD - BLUE TEAM UBUNTU v2.0 ${NC}"  
log "${BLUE}===== ${NC}"  
log "Fecha: $(date)"  
log "Host: $(hostname) | Usuario: $(whoami)"  
log "Modelo: Defensa en Profundidad - 4 Niveles"  
  
# =====  
# NIVEL 1 – RED: Perímetro y Resiliencia  
# =====  
log "\n${CYAN}┌───────────────────────────────────────────────────────────┐${NC}"  
log "${CYAN}│ NIVEL 1 – RED: Perímetro y Resiliencia │${NC}"  
log "${CYAN}└───────────────────────────────────────────────────────────┘${NC}"  
  
# 1a. Estado de servicios de red  
log "\n${GREEN}[1a] Estado de Servicios de Red${NC}"  
for s in ufw fail2ban crowdsec; do  
    if systemctl list-unit-files --quiet "${s}.service" &>/dev/null; then  
        status=$(systemctl is-active "$s" 2>/dev/null)  
        if [ "$status" = "active" ]; then  
            printf "   %-15s %b\\n" "$s" "${GREEN}✓ ACTIVO${NC}" | tee -a "$LOG_FILE"
```

```

        else
            printf "  %-15s %b\n" "$s" "${RED}X CAÍDO ($status){NC}" | tee -a
"$LOG_FILE"
            alerta "Servicio $s está CAÍDO (estado: $status)"
        fi
    else
        printf "  %-15s %b\n" "$s" "${YELLOW}△ NO INSTALADO{NC}" | tee -a
"$LOG_FILE"
        alerta "Servicio $s NO está instalado"
    fi
done

# 1b. Reglas UFW activas
log "\n${GREEN}[1b] Reglas UFW Activas{NC}"
if command -v ufw >/dev/null 2>&1 && systemctl is-active --quiet ufw; then
    sudo ufw status numbered 2>/dev/null | tee -a "$LOG_FILE"
    # Verificar que el modo por defecto sea DENY
    ufw_default=$(sudo ufw status verbose 2>/dev/null | grep "^Default:" | head -1)
    if echo "$ufw_default" | grep -qi "deny"; then
        log "${GREEN} Política por defecto: DENY ✓{NC}"
    else
        log "${RED} Política por defecto: NO es DENY{NC}"
        alerta "UFW: política por defecto no es DENY - revisar reglas"
    fi
else
    log "${YELLOW} UFW no disponible. Omitiendo.{NC}"
fi

# 1c. Puertos escuchando
log "\n${GREEN}[1c] Puertos Abiertos (TCP/UDP){NC}"
printf "  %-30s %-30s\n" "Dirección:Puerto" "Proceso" | tee -a "$LOG_FILE"
printf "  %-30s %-30s\n" "-----" "-----" | tee -a "$LOG_FILE"
sudo ss -tulpn 2>/dev/null | grep LISTEN | awk '{print $5, $7}' | \
    while IFS= read -r linea; do
        # Sanitizar: imprimir solo si contiene caracteres esperados
        if [[ "$linea" =~ ^[a-zA-Z0-9._:()\/\-\ ]+$ ]]; then
            printf "  %-30s %-30s\n" "$(echo "$linea" | awk '{print $1}')" \
                "$(echo "$linea" | awk '{print $2}')" | tee -a
"$LOG_FILE"
        fi
    done

# 1d. Verificación de protección SYN Flood (sysctl)
log "\n${GREEN}[1d] Protección SYN Flood (sysctl){NC}"
syn_cookies=$(sysctl -n net.ipv4.tcp_syncookies 2>/dev/null)
syn_backlog=$(sysctl -n net.ipv4.tcp_max_syn_backlog 2>/dev/null)
if [ "${syn_cookies:-0}" -eq 1 ]; then
    log "${GREEN} tcp_syncookies = 1 ✓ (SYN cookies activadas){NC}"
else
    log "${RED} tcp_syncookies = ${syn_cookies:-?} X DESACTIVADO{NC}"
    alerta "SYN cookies desactivadas - vulnerable a SYN flood"
fi
log " tcp_max_syn_backlog = ${syn_backlog:-?}"

# 1e. Espacio en disco (forense - evitar DoS por disco lleno)

```



```

log "\n${GREEN}[1e] Espacio en Disco (High Logging / Forense)${NC}"
printf "  %-18s %-8s %-8s %-8s\n" "Partición" "Usado" "Libre" "Uso%" | tee -a
"$LOG_FILE"
printf "  %-18s %-8s %-8s %-8s\n" "-----" "-----" "-----" "-----" | tee -a
"$LOG_FILE"
for mount in /var/log / /home; do
    if [ -d "$mount" ]; then
        read -r used avail pct _ < <(df -h "$mount" 2>/dev/null | tail -1 | awk
'{{print $3, $4, $5, $6}}')
        pct_num=$((pct//%))
        if [ "${pct_num:-0}" -ge 85 ] 2>/dev/null; then
            printf "  %-18s %-8s %-8s %b\n" "$mount" "$used" "$avail" "${RED}$pct
△${NC}" | tee -a "$LOG_FILE"
            alerta "Disco $mount al ${pct} - riesgo de DoS por logs"
        else
            printf "  %-18s %-8s %-8s %b\n" "$mount" "$used" "$avail"
"${GREEN}$pct${NC}" | tee -a "$LOG_FILE"
        fi
    fi
done

# =====
# NIVEL 2 - APLICACIÓN: Identidad Criptográfica SSH
# =====
log "\n${CYAN}|||${NC}"
log "${CYAN}|| NIVEL 2 - APLICACIÓN: Identidad Criptográfica |||${NC}"
log "${CYAN}|||${NC}"

# 2a. Configuración crítica de sshd
log "\n${GREEN}[2a] Configuración SSHD${NC}"
SSHD_CONFIG="/etc/ssh/sshd_config"
check_sshd() {
    local directiva="$1"
    local valor_esperado="$2"
    local valor_actual
    # Buscar en sshd_config y en includes (sshd_config.d/)
    valor_actual=$(sudo grep -rh "^${directiva}" /etc/ssh/sshd_config
/etc/ssh/sshd_config.d/ 2>/dev/null \
| awk '{{print $2}}' | head -1)
    if [ "${valor_actual,,}" = "${valor_esperado,,}" ]; then
        printf "  %-30s %b\n" "$directiva" "${GREEN}${valor_actual} ✓${NC}" | tee -a
"$LOG_FILE"
    else
        printf "  %-30s %b\n" "$directiva" "${RED}${valor_actual:-no configurado} X
(esperado: $valor_esperado)${NC}" | tee -a "$LOG_FILE"
        alerta "SSHD: $directiva no es '$valor_esperado' (valor actual:
${valor_actual:-vacío})"
    fi
}

check_sshd "PasswordAuthentication" "no"
check_sshd "PermitRootLogin" "no"
check_sshd "PubkeyAuthentication" "yes"
check_sshd "PermitEmptyPasswords" "no"
check_sshd "X11Forwarding" "no"

```

```

check_sshd "MaxAuthTries" "3"

# 2b. Verificar que los usuarios con acceso SSH usan ED25519
log "\n${GREEN}[2b] Llaves SSH Autorizadas por Usuario${NC}"
printf " %-15s %-12s %-45s\n" "Usuario" "Tipo llave" "Huella" | tee -a "$LOG_FILE"
printf " %-15s %-12s %-45s\n" "-----" "-----" "-----" | tee -a "$LOG_FILE"

while IFS=: read -r usuario _ uid _ _ home _; do
    (( uid >= 1000 )) || continue # solo usuarios regulares
    auth_keys="${home}/.ssh/authorized_keys"
    [ -f "$auth_keys" ] || continue
    while IFS= read -r linea; do
        tipo=$(echo "$linea" | awk '{print $1}')
        # Sanitizar: solo tipos de llave conocidos
        if [[ "$tipo" =~ ^(ssh-ed25519|ssh-rsa|ecdsa-sha2-nistp256|ssh-dss)$ ]]; then
            huella=$(echo "$linea" | ssh-keygen -l -f /dev/stdin 2>/dev/null | awk
            '{print $2}')
            if [ "$tipo" = "ssh-ed25519" ]; then
                printf " %-15s %b %-45s\n" "$usuario" "${GREEN}$tipo ✓${NC}"
                "${huella:-n/a}" | tee -a "$LOG_FILE"
            else
                printf " %-15s %b %-45s\n" "$usuario" "${YELLOW}$tipo △${NC}"
                "${huella:-n/a}" | tee -a "$LOG_FILE"
                alerta "Usuario $usuario usa $tipo (recomendado: ED25519)"
            fi
        fi
    done < "$auth_keys"
done < /etc/passwd

# 2c. Permisos en ~/.ssh
log "\n${GREEN}[2c] Permisos Directorio .ssh${NC}"
while IFS=: read -r usuario _ uid _ _ home _; do
    (( uid >= 1000 )) || continue
    ssh_dir="${home}/.ssh"
    [ -d "$ssh_dir" ] || continue
    perms=$(stat -c "%a" "$ssh_dir" 2>/dev/null)
    if [ "$perms" = "700" ]; then
        printf " %-20s %b\n" "$ssh_dir" "${GREEN}${perms} ✓${NC}" | tee -a
"$LOG_FILE"
    else
        printf " %-20s %b\n" "$ssh_dir" "${RED}${perms} X (debe ser 700)${NC}" | tee
-a "$LOG_FILE"
        alerta "Permisos inseguros en $ssh_dir ($perms) - debe ser 700"
    fi
done < /etc/passwd

# =====
# NIVEL 3 - COMPORTAMIENTO: Defensa Colaborativa e IPS
# =====
log "\n${CYAN} [NIVEL 3 - COMPORTAMIENTO: Defensa Colaborativa e IPS]${NC}"
log "${CYAN} [NIVEL 3 - COMPORTAMIENTO: IPS y Colaboración]${NC}"
log "${CYAN} [NIVEL 3 - COMPORTAMIENTO: IPS y Colaboración]${NC}"

# 3a. Fail2Ban - jails con sanitización
log "\n${GREEN}[3a] Fail2Ban: IPs Baneadas Actuales${NC}"

```

```
if command -v fail2ban-client &>/dev/null && systemctl is-active --quiet fail2ban;  
then  
    printf "   %-25s %-12s\n" "Jaula" "Baneos activos" | tee -a "$LOG_FILE"  
    printf "   %-25s %-12s\n" "-----" "-----" | tee -a "$LOG_FILE"  
  
    mapfile -t jail_list <<(  
        sudo fail2ban-client status 2>/dev/null \  
            | grep "Jail list" \  
            | sed 's/. *list://' \  
            | tr ',' '\n' \  
            | tr -d ' \t' \  
            | grep -v '^$'  
    )  
  
    if [ "${#jail_list[@]}" -eq 0 ]; then  
        log "${YELLOW} No se encontraron jaulas activas.${NC}"  
    else  
        for jail in "${jail_list[@]"; do  
            # — SANITIZACIÓN: solo alfanuméricos, guiones y guiones bajos —  
            if ! validar_jail "$jail"; then  
                log "${RED} Nombre de jaula inválido ignorado: '$jail'${NC}"  
                alerta "Nombre de jaula sospechoso detectado: '$jail'"  
                continue  
            fi  
            count=$(sudo fail2ban-client status "$jail" 2>/dev/null \  
                    | grep "Currently banned" \  
                    | awk '{print $NF}')  
            count=${count:-0}  
            if [ "$count" -gt 0 ] 2>/dev/null; then  
                printf "   %-25s %b\n" "$jail" "${RED}$count △${NC}" | tee -a  
"$LOG_FILE"  
                alerta "Fail2Ban: $count IPs baneadas en jaula '$jail'"  
            else  
                printf "   %-25s %b\n" "$jail" "${GREEN}$count${NC}" | tee -a  
"$LOG_FILE"  
            fi  
        done  
    fi  
else  
    log "${YELLOW} fail2ban no está instalado o no está corriendo.${NC}"  
    alerta "Fail2Ban no está activo"  
fi  
  
# 3b. CrowdSec – últimas decisiones + estado del Hub  
log "\n${GREEN}[3b] CrowdSec: Últimas 5 Decisiones${NC}"  
if command -v cscli &>/dev/null && systemctl is-active --quiet crowdsec; then  
    sudo cscli decisions list -l 5 --no-color 2>/dev/null \  
        | grep -v "^[:space:]]*$\|—|\|=\\||_|`||_||^\\s*ID\s" \  
        | while IFS= read -r linea; do  
            if [[ "$linea" =~ ^[a-zA-Z0-9./:_\ |\-\+]+$ ]]; then  
                echo " $linea" | tee -a "$LOG_FILE"  
            fi  
        done  
    # Contar total de decisiones activas
```



```

total=$(sudo cscli decisions list --no-color 2>/dev/null | grep -c "^|" || echo
"0")
log " Total de decisiones activas: ${total}"

# Verificar escenarios del Hub con actualizaciones disponibles
log "\n${GREEN}[3b+] CrowdSec Hub: Estado de Escenarios${NC}"
hub_updates=$(sudo cscli hub list --no-color 2>/dev/null \
| grep -i "update available" | wc -l)
if [ "${hub_updates}" -gt 0 ]; then
    log "${YELLOW} $hub_updates escenario(s) con actualización disponible:${NC}"
    sudo cscli hub list --no-color 2>/dev/null \
    | grep -i "update available" | sed 's/^/ /' | tee -a "$LOG_FILE"
    alerta "CrowdSec Hub: $hub_updates escenario(s) desactualizados - ejecutar:
sudo cscli hub update && sudo cscli hub upgrade"
else
    log "${GREEN} Todos los escenarios del Hub están actualizados. ✓${NC}"
fi
else
    log "${YELLOW} CrowdSec no está instalado o no está corriendo.${NC}"
    alerta "CrowdSec no está activo"
fi

# 3c. Intentos de fuerza bruta SSH - con validación de IPs
log "\n${GREEN}[3c] Top 5 IPs con Intentos SSH Fallidos (Últimas 24h)${NC}"
SSH_LOGS=$(sudo journalctl -u ssh -u sshd --since "24 hours ago" 2>/dev/null)

if [ -z "$SSH_LOGS" ]; then
    log "${YELLOW} No se encontraron logs SSH en las últimas 24h.${NC}"
else
    # Extraer IPs brutas con regex
    IPS_BRUTAS=$(echo "$SSH_LOGS" \
    | grep -i "Failed password\|authentication failure" \
    | grep -oP '(?<=from )\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}')

    if [ -z "$IPS_BRUTAS" ]; then
        log "${GREEN} Sin intentos fallidos registrados. ✓${NC}"
    else
        # — SANITIZACIÓN: validar cada IP antes de imprimir —
        RESULT=$(echo "$IPS_BRUTAS" | sort | uniq -c | sort -nr | head -n 5)
        printf " %-10s %-20s\n" "Intentos" "IP" | tee -a "$LOG_FILE"
        printf " %-10s %-20s\n" "-----" "---" | tee -a "$LOG_FILE"
        while read -r count ip; do
            if validar_ip "$ip"; then
                printf " %-10s %b\n" "$count" "${RED}$ip${NC}" | tee -a "$LOG_FILE"
                if [ "$count" -ge 20 ]; then
                    alerta "IP $ip con $count intentos SSH - considerar baneo manual"
                fi
            else
                log "${YELLOW} IP con formato inválido ignorada: '$ip'${NC}"
                alerta "IP con formato sospechoso en logs SSH: '$ip'"
            fi
        done <<< "$RESULT"
    fi
fi
fi

```



```

# =====
# NIVEL 4 – HOST: Integridad y Auditoría
# =====
log "\n${CYAN}"
log "${CYAN}" NIVEL 4 – HOST: Integridad y Auditoría "${NC}"
log "${CYAN}" "${NC}"

# 4a. RKHunter – escaneo completo 1x/día; el resto de ejecuciones lee el log
log "\n${GREEN}[4a] RKHunter: Escaneo de Rootkits${NC}"
if command -v rkhunter &>/dev/null; then
    RKHUNTER_LOG="/var/log/rkhunter_$(date +%F).log"

    if [ ! -f "$RKHUNTER_LOG" ]; then
        # Primera ejecución del día: actualizar base de datos y correr check completo
        log " Ejecutando escaneo completo (primera vez hoy)..."
        sudo rkhunter --update --nocolors 2>/dev/null \
            | grep -E "Updated|up-to-date" | sed 's/^/ /' | tee -a "$LOG_FILE" ||
true
        sudo rkhunter --check --nocolors --skip-keypress \
            --logfile "$RKHUNTER_LOG" 2>/dev/null
    else
        # Ejecuciones intermedias: leer log existente sin relanzar el check
        log " Log del día ya existe – leyendo resultados previos (sin relanzar
escaneo)."
        log " Log: ${RKHUNTER_LOG}"
    fi

    # Extraer resumen del log (igual en ambos casos)
    warnings=$(grep -c "\[ Warning \]" "$RKHUNTER_LOG" 2>/dev/null || echo "0")
    found=$(grep -c "\[ Found \]" "$RKHUNTER_LOG" 2>/dev/null || echo "0")
    if [ "$warnings" -gt 0 ] || [ "$found" -gt 0 ]; then
        log "${RED} Advertencias: $warnings | Encontrados: $found${NC}"
        log "${YELLOW} Ver detalle en: $RKHUNTER_LOG${NC}"
        alerta "RKHunter: $warnings advertencias, $found hallazgos – revisar
$RKHUNTER_LOG"
        grep "\[ Warning \]|\[ Found \]" "$RKHUNTER_LOG" 2>/dev/null | head -10 | \
            sed 's/^/ /' | tee -a "$LOG_FILE"
    else
        log "${GREEN} Sin advertencias ni hallazgos. ✓${NC}"
    fi
else
    log "${YELLOW} rkhunter no instalado. Instala con: sudo apt install
rkhunter${NC}"
    alerta "RKHunter no está instalado"
fi

# 4b. Lynis – ejecutar auditoría y reportar score
log "\n${GREEN}[4b] Lynis: Auditoría del Sistema${NC}"
if command -v lynis &>/dev/null; then
    LYNIS_LOG="/var/log/lynis_$(date +%F).log"
    sudo lynis audit system --quiet --no-colors --logfile "$LYNIS_LOG" 2>/dev/null ||
true

    # Extraer hardening index

```

```

score=$(grep "Hardening index" "$LYNIS_LOG" 2>/dev/null | awk '{print $NF}' | tr -
d '[]')
warnings_lynis=$(grep -c "^Warning" "$LYNIS_LOG" 2>/dev/null || echo "0")
suggestions=$(grep -c "^Suggestion" "$LYNIS_LOG" 2>/dev/null || echo "0")

if [ -n "$score" ]; then
    if [ "$score" -ge 80 ]; then
        log "${GREEN} Hardening index: ${score}/100 ✓${NC}"
    elif [ "$score" -ge 60 ]; then
        log "${YELLOW} Hardening index: ${score}/100 ⚠ (objetivo: ≥80)${NC}"
        alerta "Lynis hardening index $score/100 - por debajo del objetivo"
    else
        log "${RED} Hardening index: ${score}/100 ✗ CRÍTICO${NC}"
        alerta "Lynis hardening index $score/100 - CRÍTICO"
    fi
fi
log " Advertencias: ${warnings_lynis} | Sugerencias: ${suggestions}"
log "${YELLOW} Ver detalle en: $LYNIS_LOG${NC}"

# Mostrar top 5 advertencias
if [ "$warnings_lynis" -gt 0 ]; then
    log "\n Top advertencias Lynis:"
    grep "^Warning" "$LYNIS_LOG" 2>/dev/null | head -5 | sed 's/^/ /' | tee -a
"$LOG_FILE"
fi
else
    log "${YELLOW} lynis no instalado. Instala con: sudo apt install lynis${NC}"
    alerta "Lynis no está instalado"
fi

# 4c. Binarios críticos modificados recientemente (últimas 24h) + atributo immutable
log "\n${GREEN}[4c] Binarios Críticos: Modificaciones y Atributo Inmutable${NC}"
BINARIOS_CRITICOS=(
    /bin/bash /bin/sh /usr/bin/sudo /usr/bin/passwd
    /usr/sbin/sshd /usr/bin/ssh /sbin/iptables
)
encontrados=0
for bin in "${BINARIOS_CRITICOS[@]"; do
    [ -f "$bin" ] || continue

    # Verificar modificación por mtime
    if find "$bin" -mtime -1 2>/dev/null | grep -q .; then
        log "${RED} MODIFICADO (mtime): $bin${NC}"
        alerta "Binario crítico modificado en las últimas 24h: $bin"
        encontrados=$((encontrados + 1))
    fi

    # Verificar atributo immutable con lsattr
    # Un binario protegido debe mostrar 'i' en los atributos.
    # Si NO tiene 'i', o si lsattr falla (sistema de archivos no lo soporta),
    # se reporta como advertencia -no alerta crítica- para evitar falsos positivos.
    if command -v lsattr &>/dev/null; then
        attrs=$(lsattr "$bin" 2>/dev/null | awk '{print $1}')
        if [ -z "$attrs" ]; then
            # lsattr no pudo leer el archivo (fs no soportado, etc.)

```

```

        printf "  %-35s %b\n" "$bin" "${YELLOW}lsattr no disponible en este
fs${NC}" \
        | tee -a "$LOG_FILE"
    elif echo "$attrs" | grep -q 'i'; then
        printf "  %-35s %b\n" "$bin" "${GREEN}immutable ✓${NC}" | tee -a
"$LOG_FILE"
    else
        printf "  %-35s %b\n" "$bin" "${YELLOW}sin atributo immutable △${NC}" \
        | tee -a "$LOG_FILE"
        # Solo alerta si además fue modificado recientemente (combinación
sospechosa)
        if find "$bin" -mtime -1 2>/dev/null | grep -q .; then
            alerta "Binario $bin: modificado Y sin bit immutable - altamente
sospechoso"
        fi
    fi
done
if [ "$encontrados" -eq 0 ]; then
    log "${GREEN} Sin modificaciones de mtime en binarios críticos. ✓${NC}"
fi

# 4d. Usuarios con shell válido (posibles backdoors)
log "\n${GREEN}[4d] Usuarios con Shell Válido${NC}"
printf "  %-15s %-8s %-20s\n" "Usuario" "UID" "Shell" | tee -a "$LOG_FILE"
printf "  %-15s %-8s %-20s\n" "-----" "----" "-----" | tee -a "$LOG_FILE"
while IFS=: read -r usuario _ uid _ _ shell; do
    # Solo mostrar shells ejecutables (excluir /sbin/nologin, /bin/false)
    if [[ "$shell" != */nologin && "$shell" != */false && "$shell" != "" ]]; then
        if (( uid == 0 )); then
            printf "  %-15s %-8s %b\n" "$usuario" "$uid" "${YELLOW}$shell (root)${NC}"
| tee -a "$LOG_FILE"
        elif (( uid >= 1000 )); then
            printf "  %-15s %-8s %b\n" "$usuario" "$uid" "${GREEN}$shell${NC}" | tee -
a "$LOG_FILE"
        else
            # Usuario de sistema con shell - sospechoso
            printf "  %-15s %-8s %b\n" "$usuario" "$uid" "${RED}$shell △ (sistema con
shell)${NC}" | tee -a "$LOG_FILE"
            alerta "Usuario de sistema '$usuario' (uid=$uid) tiene shell: $shell"
        fi
    fi
done < /etc/passwd

# =====
# RESUMEN FINAL Y ALERTAS
# =====
log "\n${BLUE}===== ${NC}"
log "${YELLOW} RESUMEN EJECUTIVO${NC}"
log "${BLUE}===== ${NC}"

if [ "$ALERTAS" -eq 0 ]; then
    log "${GREEN} Estado general: ÓPTIMO - 0 alertas críticas ✓${NC}"
else

```

```

log "${RED} Estado general: ATENCIÓN - ${ALERTAS} alerta(s) detectada(s) X${NC}"
log ""
log "${YELLOW} Hallazgos:${NC}"
for msg in "${ALERTAS_MSG[@]}"; do
    log " ${RED}•${NC} $msg"
done
fi

log "\n Niveles evaluados:"
log "  ${CYAN}[1]${NC} Red          - UFW, puertos, SYN flood, disco"
log "  ${CYAN}[2]${NC} Aplicación - SSHD config, llaves ED25519, permisos"
log "  ${CYAN}[3]${NC} Comportamiento - Fail2Ban, CrowdSec, brute-force SSH"
log "  ${CYAN}[4]${NC} Host          - RKHunter, Lynis, binarios, usuarios"

log "\n${GREEN} Log completo guardado en: ${YELLOW}${LOG_FILE}${NC}"
log "${BLUE}===== ${NC}"

# Disparar alertas externas solo si hay hallazgos
if [ "$ALERTAS" -gt 0 ]; then
    enviar_webhook
    enviar_email
fi

exit 0

```


23. Port Knocking ¿Porque no está en esta guía?

Esa es una pregunta excelente que toca el debate entre **seguridad por oscuridad** y **seguridad por diseño**. En un entorno empresarial donde ya existe una **VPN**, el Port Knocking cambia de rol: pasa de ser una necesidad a ser una *"capa de paranoia extra"*.

Aquí tenemos el análisis de si es bueno o malo integrarlo:

¿Qué es el Port Knocking?

Es como un *"saludo secreto"*. El puerto SSH (22) está cerrado para todo el mundo en el firewall. Para que se abra, debemos enviar una secuencia específica de paquetes a otros puertos cerrados (por ejemplo: un toque al 7000, otro al 8000 y otro al 9000). Solo entonces, el firewall abre el puerto 22 exclusivamente para nuestra IP durante unos segundos.

Port Knocking vs. VPN: ¿Cuál gana?

En una empresa, la **VPN** es la reina por una razón: **cifrado y autenticación**.

- **VPN:** Crea un túnel seguro. Nadie puede ver lo que viaja dentro. Requiere usuario, contraseña y, idealmente, MFA (Token).
- **Port Knocking:** Solo **oculta** el puerto. No cifra nada y no identifica quién eres, solo identifica que "alguien conoce el código".

¿Es bueno incorporarlo con una VPN?

- **Lo Bueno:** Si un atacante descubre una vulnerabilidad en nuestro software de VPN (como ha pasado con Fortinet o Pulse Secure), el Port Knocking evitaría que el atacante siquiera vea que el servicio de VPN está ahí. Es la **invisibilidad total**.
- **Lo Malo (El problema real):** Es frágil. Si la red tiene latencia o los paquetes llegan desordenados, el "toque" falla y nos quedamos fuera. Además, es difícil de gestionar para 50 empleados; todos tendrían que instalar un software extra para *"tocar la puerta"* antes de conectar la VPN.

El veredicto: ¿Cuándo usarlo?

Escenario	¿Usar Port Knocking?	Razón
Usuario hogar / Admin solo	SÍ (Recomendado)	Es divertido, aprendes mucho y te da una capa de invisibilidad brutal sin costo.
Empresa con VPN	NO (Normalmente)	Añade demasiada complejidad y puntos de fallo. Si la VPN cae, el soporte técnico se vuelve una pesadilla.
Servidor de respaldo crítico	SÍ (Como último recurso)	Si todo falla y la VPN es hackeada, tener un acceso oculto por Port Knocking puede salvarte la vida.

¿Qué es mejor que el Port Knocking en una empresa?

En lugar de Port Knocking, las empresas modernas están usando algo llamado **Single Packet Authorization (SPA)** (como *fwknop*). Es la evolución:

1. Envías un **único paquete cifrado** que contiene tu identidad.
2. El firewall lo descifra, verifica que eres tú y te abre la puerta.
3. Es mucho más rápido, seguro y difícil de detectar que el Port Knocking tradicional.

Resumen para nuestro caso:

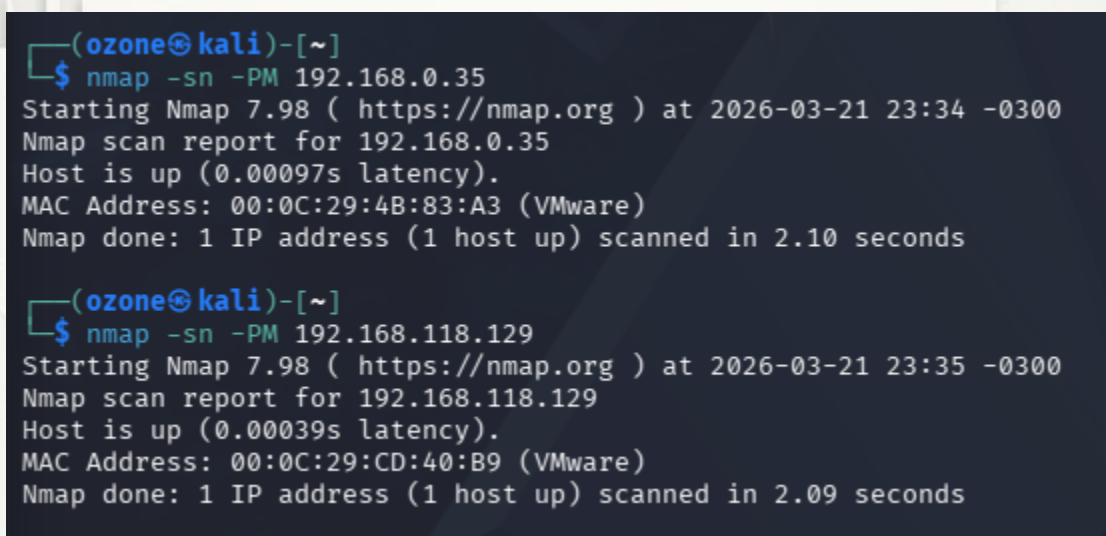
Si ya tenemos el **hardening de UFW** y **CrowdSec**, poner una **VPN** es el estándar de oro profesional (esta guía no cubre la implementación de una VPN). El Port Knocking sería como poner una puerta camuflada detrás de un muro de hormigón: muy seguro, pero quizás exagerado si ya tenemos guardias (CrowdSec) vigilando.

23. SEGURIDAD V/S FUNCIONALIDAD

El dilema de la seguridad. Podría parecer una buena idea “Bloquear TODO” pero esto no es una buena idea y a continuación veremos porqué.

En seguridad existe un concepto llamado “Seguridad por Oscuridad” que básicamente es, me escondo para que no me vean y por lo tanto **podría** no ser atacado; llevado a la práctica, el bloqueo de ICMP es una muestra de ello. Al bloquear ICMP no debiésemos aparecer en los escaneos.

Como se puede ver en la siguiente imagen un escaneo de nmap que utiliza una técnica “antigua” ha pasado de nuestras defensas y ha obtenido respuesta.



```
(ozone@kali)-[~]
$ nmap -sn -PM 192.168.0.35
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-21 23:34 -0300
Nmap scan report for 192.168.0.35
Host is up (0.00097s latency).
MAC Address: 00:0C:29:4B:83:A3 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 2.10 seconds

(ozone@kali)-[~]
$ nmap -sn -PM 192.168.118.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-21 23:35 -0300
Nmap scan report for 192.168.118.129
Host is up (0.00039s latency).
MAC Address: 00:0C:29:CD:40:B9 (VMware)
Nmap done: 1 IP address (1 host up) scanned in 2.09 seconds
```

En el punto 15.2 de esta guía nosotros instalamos la colección de reglas de crowdsecurity/iptables,

Esa colección es **el cerebro**, pero necesita que el firewall (**UFW**) sea sus "**ojos**". Aquí explico por qué el escaneo **-PM (Address Mask)** pasó a pesar de tener instalada la colección:

El problema de la "ceguera" de Logs

Debemos tener claro que CrowdSec no adivina los ataques; los lee de los logs.

- La colección que instalamos busca patrones como: *"Si esta IP intentó tocar 10 puertos cerrados en 1 minuto, banéala"*.
- **Pero hay un truco:** UFW, por defecto, **no guarda logs de los paquetes ICMP** (pings, address-mask, etc.).
- Como UFW no escribió nada en `/var/log/ufw.log` sobre ese paquete -PM, CrowdSec nunca se enteró de que Kali estaba "saludando".

¿Para qué sirve entonces la colección iptables?

Esa colección es excelente para detectar:

- **Escaneo de puertos TCP/UDP:** Cuando Nmap busca el puerto 22, 80, 443, etc.
- **Ataques de inundación (Flooding):** Si intentan colapsar nuestra red.
- **Escaneos sigilosos (Stealth Scans):** Que UFW bloquea y anota en el log.

El "hueco" en nuestra defensa que encontró nmap con -PM

Al usar -PM, usamos un paquete que el kernel de Linux suele responder de forma automática y sobre todo, **silenciosa**.

- **UFW:** Lo dejó pasar porque no había una regla de DROP específica para el Tipo 17 (Address Mask).
- **CrowdSec:** No vio nada en los logs, así que no pudo banear a Kali.

¿La solución maestra? ¿¿¿(Hardening Total)???

Para que esto no vuelva a pasar, debemos hacer dos cosas:

A. Bloquear el tipo de paquete en UFW: Esto hace que el servidor sea invisible al -PM.

B. Asegurarnos de que UFW loguee el bloqueo: Si queremos que CrowdSec banee a alguien solo por intentar hacerte un barrido de pings, necesitamos que UFW anote esos bloqueos.

PERO AQUÍ EL GRAN "PERO" Y LA CAPACIDAD DEL ADMINISTRADOR DE SISTEMA DE TOMAR LA DECISIÓN.

Advertencia de "Ruido": Si activamos logs para **TODO** el **ICMP**, nuestro archivo `/var/log/ufw.log` crecerá muy rápido si alguien nos hace un ataque de ping. Por eso, en servidores de producción, preferimos simplemente **bloquearlo en silencio (DROP)** y dejar que CrowdSec se encargue ***de los escaneos de puertos, que son los que realmente preceden a un hackeo.***

Entonces ahora que entendemos este tipo de escenarios podemos hablar del título de este capítulo, **Seguridad v/s Funcionalidad.**

A continuación entendemos las razones sobre cuándo y porqué **"demasiado bloqueo"** se vuelve un problema:

Reconocimiento no es lo mismo que Intrusión

Un escaneo como `-PM` (Address Mask) es como si alguien pasara por la calle mirando si tenemos las luces encendidas.

- Saber que existes (Reconocimiento) no es lo mismo que entrar a la casa (Explotación).
- CrowdSec y nuestro hardening de puertos (SSH, Web, etc.) son los que realmente impiden que el atacante entre.

Los problemas de bloquear "todo"

Si nos obsesionamos con bloquear cada variante de ICMP que Nmap invente, podríamos causar estos dolores de cabeza:

- **Problemas de conectividad intermitente:** Como el "MTU Path Discovery" algunas webs no cargarán o irán lentas.
- **Falsos positivos en CrowdSec:** Si bloqueamos y logueamos cada pequeño paquete de red local, podríamos terminar baneando por error a nuestro propio Router o a nuestra impresora, quedando nosotros mismo fuera de la red.
- **Dificultad para soporte:** Si un día nuestro internet falla y llamas al proveedor, ellos no podrán hacerle un "ping" a nuestro equipo para saber si el cable está bien.

Entonces, ¿qué es lo que SÍ VALE LA PENA?

En seguridad seguimos la regla del **80/20**:

- **El 80% de la protección** viene de tener los puertos cerrados (UFW) y banear a quien intente entrar por la fuerza (CrowdSec).
- **El otro 20%** (bloquear -PM, -PP, etc.) es "decoración". Es bueno tenerlo, pero si te causa problemas, es mejor dejarlo como está.

Nuestra configuración actual... ¿es Excelente?

Si, miremos lo que hemos logrado:

1. **Invisibilidad al ping Normal (-PE)**: Los bots básicos de internet no nos verán.
2. **Puertos protegidos**: Nadie puede entrar sin que CrowdSec lo note.
3. **Logs activos**: Tenemos visibilidad de lo que pasa.

Veredicto final:

Si este servidor es para aprender o para uso personal/hogareño, **dejarlo como está**. Ya tenemos un nivel de seguridad superior al 95% de los servidores domésticos. El hecho de que Nmap necesite usar banderas "raras" como -PM para verte ya es una señal de que tu firewall está haciendo un gran trabajo.

¿Pero si fuese algo real, una pequeña o mediana empresa?

Para responder a esto debemos colocarnos el sombrero de **Consultor de ciberseguridad**, y la respuesta cambia un poco cuando hay dinero, reputación y datos de clientes de por medio.

En una **PyME**, el enfoque de *"déjalo así"* no es suficiente. Aquí explico por qué en un entorno profesional sí querríamos dar el siguiente paso, pero de una forma distinta:

El riesgo de "Reconocimiento = Objetivo"

Para un atacante profesional, encontrar un servidor que responde a -PM pero no a -PE (ping normal) es una señal clara de: *"Aquí hay alguien que intentó protegerse pero dejó la ventana entreabierta"*.

- En una empresa, la **Ofuscación** (hacerse el invisible) es una capa **real** de defensa. Si el escáner del atacante dice "Host down", lo más probable es que pase a la siguiente IP de la lista. **Si no te ven, no te atacan.**

El principio de "Denegación por defecto"

En una empresa seria, la regla de oro es: **Todo lo que no sea estrictamente necesario para el negocio, se bloquea.**

- *¿Necesita el servidor contable responder a un Address Mask Request de ICMP para que la empresa funcione?* **No.** Entonces se bloquea.
- *¿Necesita el servidor web responder a pings desde China?* **No.** Entonces se bloquea (o se limita a la IP de la oficina).

La ESTRATEGIA PROFESIONAL: El "Perímetro"

En una PyME, no confiaríamos (ni debiésemos tener) solo en el **UFW** de un servidor. Tendríamos un **Firewall Perimetral** (como un Fortigate, pfSense o Cisco) delante de todo.

- **Firewall de red:** Bloquearía **TODO** el ICMP "raro" (-PM, -PP, etc.) **antes** de que llegue a nuestro servidor Ubuntu.
- **UFW del Ubuntu:** Sería nuestra segunda línea de defensa (Defensa en profundidad).
- **CrowdSec:** Sería el sistema que banea automáticamente si alguien logra saltarse el firewall perimetral y empieza a escanear los puertos internos.

Lo que SÍ aplica de lo que hemos hecho:

Incluso en una empresa de 500 empleados, lo que hemos configurado es la base de todo:

1. **Logs centralizados:** En una empresa, esos logs de **UFW** que estamos viendo irían a un sistema llamado **SIEM** (como Wazuh, ELK o Splunk) para que los analistas vean los ataques en tiempo real.FDD
2. **IPS activo (CrowdSec):** Es vital. Ninguna empresa puede permitirse tener a un humano bloqueando IPs manualmente un sábado a las 3 AM.

Veredicto final "Versión profesional":

Si esto fuera para una **PyME real**, la recomendación sería:

1. **Cierra los huecos ICMP en el firewall perimetral:** Añade esas reglas para **-PM** y **-PP**. Es mejor pecar de paranoico que de permisivo.
2. **Configura una VPN:** En lugar de tener el puerto 22 (**SSH**) abierto al mundo (aunque CrowdSec lo proteja), lo cerraríamos por completo en **UFW** y solo permitiríamos entrar a través de una **VPN** (como *OpenVPN* o *Wireguard*).
3. **Mantenimiento:** No basta con instalar CrowdSec; hay que revisar semanalmente qué IPs está baneando para entender de dónde vienen los ataques.

24. Comandos (Actualización Lab 3)

Fase 1: HIGIENE BASE Y AUTOMATIZACIÓN

Antes de blindar, el sistema debe estar limpio y actualizado.

Comando	Función
<code>sudo apt update && sudo apt upgrade -y</code>	Actualización del sistema: Sincronizar repositorios y aplicar parches.
<code>sudo apt autoremove -y</code>	Limpieza de dependencias: Eliminar paquetes "huérfanos" que amplían la superficie de ataque.
<code>sudo apt install unattended-upgrades</code>	Parcheo Automático: Instalar herramienta para actualizaciones ante riesgos críticos.
<code>sudo dpkg-reconfigure unattended-upgrades</code>	Configurar unattended-upgrades para que el sistema se actualice solo.
<code>systemctl status unattended-upgrades</code>	Verificación del estado del parcheo automático.

Fase 2: GESTIÓN DE IDENTIDADES Y PRIVILEGIOS

El objetivo es eliminar el uso de la cuenta root para tareas diarias y restringir el acceso.

Comando	Función
<code>sudo adduser <nombre_usuario></code>	Creación de usuario administrativo: No trabajar directamente como root.
<code>sudo usermod -aG sudo <nombre_usuario></code>	Agregar el nuevo usuario al grupo sudo para gestión de privilegios.
<code>cut -d: -f1 /etc/passwd</code>	Auditoría de cuentas: Listar usuarios que tienen acceso al sistema.
<code>getent group sudo</code>	Revisar qué usuarios tienen privilegios administrativos (sudo).

Fase 3: HARDENING DE SSH

SSH es el vector principal de ataque. Se debe migrar de contraseñas a llaves criptográficas.

Comando / Parámetro	Función
<code>ssh-keygen -t ed25519</code>	Generación de llaves (en cliente): Usar algoritmos modernos como ED25519.
<code>PermitRootLogin no</code>	Configuración del servidor: Deshabilitar acceso directo al usuario root.
<code>PasswordAuthentication no</code>	Configuración del servidor: Permitir solo acceso mediante llaves criptográficas.
<code>LoginGraceTime 30</code>	Configuración del servidor: Reducir el tiempo de espera para el inicio de sesión.
<code>MaxAuthTries 3</code>	Configuración del servidor: Limitar el número de intentos de autenticación.
<code>Banner /etc/issue.net</code>	Configuración del servidor: Establecer advertencia legal disuasoria.
<code>sudo systemctl restart ssh</code>	Reinicio preventivo para aplicar los cambios en la configuración.

Fase 4: FIREWALL (UFW)

Configuración del firewall bajo el Principio de Mínimo Privilegio.

Comando	Función
<code>sudo ufw reset</code>	Reset del firewall: Eliminar configuraciones previas.
<code>sudo ufw default deny incoming</code>	Política por defecto: Denegar todo el tráfico entrante.
<code>sudo ufw default allow outgoing</code>	Política por defecto: Permitir todo el tráfico saliente.
<code>sudo ufw allow 22/tcp</code>	Apertura quirúrgica: Abrir solo puerto necesario (ej. SSH).
<code>sudo ufw limit ssh</code>	Aplicar protección básica contra ataques de fuerza bruta.
<code>/etc/ufw/before.rules</code>	este archivo contiene las reglas de iptables que se ejecutan antes de cualquier regla que hayamos configurado manualmente. Es una pieza crítica si estás administrando un servidor Linux porque define el comportamiento base del firewall.
<code>lvextend, resize2fs</code>	Expandir disco con LVM para evitar saturación por registros.
<code>sudo ufw logging high</code>	Activar logs de nivel alto para mayor visibilidad.

Fase 5: DEFENSA ACTIVA (FAIL2BAN + CROWDSEC)

Pasamos de un firewall estático a una defensa reactiva e inteligente.

Comando	Función
<code>sudo apt install fail2ban</code>	<i>Fail2Ban: Instalación de la herramienta.</i>
<code>sshd, sshd-aggressive, recidive</code>	<i>Configuración de jaulas: Activar protección para ataques persistentes y reincidentes. Ver capítulo.</i>
<code>sudo fail2ban-client status <jail_name></code>	<i>Gestión: Revisar el estado y las IPs bloqueadas por la cárcel especificada.</i>
<code>sudo apt install *** crowdsec crowdsec-firewall-bouncer-iptables ***</code>	<i>CrowdSec: Instalación del motor y bouncer. Ver capítulo.</i>
<code>sudo cscli decisions list</code>	<i>Ver IPs bloqueadas por la red global de inteligencia.</i>
<code>sudo cscli metrics</code>	<i>Estadísticas: Revisar el rendimiento y detección del motor de CrowdSec.</i>

Fase 6: INTEGRIDAD Y AUDITORÍA (LYNIS + RKHUNTER)

Vigilancia interna para detectar intrusiones que hayan saltado el perímetro.

Comando	Función
<code>sudo rkhunter --update</code>	<i>RKHunter (Rootkit Hunter): Actualizar la base de datos de firmas.</i>
<code>sudo rkhunter --propupd</code>	<i>Establecer línea base (Baseline) de las propiedades de los archivos.</i>
<code>sudo rkhunter --check</code>	<i>Ejecución de escaneo completo en busca de rootkits.</i>
<code>sudo lynis audit system</code>	<i>Lynis: Ejecución de auditoría profesional de seguridad del sistema.</i>
<code>sudo grep "warning[]" /var/log/lynis-report.dat</code>	<i>Revisión selectiva de advertencias detectadas por Lynis.</i>
<code>find / -perm -4000 -type f 2>/dev/null</code>	<i>Auditoría de Binarios SUID: Buscar archivos que permitan escalada de privilegios.</i>

Fase 7: MONITOREO Y FORENSE

Comando	Función
<code>ss -tulpn</code>	Sockets activos: Revisar qué servicios están escuchando realmente en la red.
<code>sudo chmod 640 /var/log/auth.log</code>	Permisos de logs: Asegurar que solo root pueda leer evidencias críticas.
<code>tail -f /var/log/auth.log</code>	Vigilancia en tiempo real: Monitorear intentos de acceso al sistema.
<code>tail -f /var/log/ufw.log</code>	Vigilancia en tiempo real: Monitorear bloqueos de red realizados por el firewall.

25. Checklists

25.1. Checklist de mantención y monitoreo continuo

Este checklist debe ejecutarse de forma periódica (semanal/mensual) para asegurar que la "fortaleza" siga siendo impenetrable.

Acción	Explicación	Comando
Higiene de parches	Verificar que el sistema esté al día y que las actualizaciones desatendidas funcionen.	<code>sudo apt update && sudo apt upgrade -s (simulación) o revisar /var/log/unattended-upgrades/</code>
Salud de la inteligencia de amenazas	Confirmar que CrowdSec esté recibiendo señales y el bouncer esté activo.	<code>sudo cscli metrics y sudo cscli bouncers list</code>
Estado de las jails	Asegurar que Fail2Ban esté protegiendo activamente los servicios clave (sshd, recidive, portscan).	<code>sudo fail2ban-client status</code>
Integridad de base de datos de RKHunter	Tras cualquier actualización de software legítima, actualizar la firma de archivos.	<code>sudo rkhunter --propupd</code>
Verificación de puertos	Confirmar que no hay servicios nuevos "escuchando" innecesariamente.	<code>ss -tulpn</code>
Monitoreo de espacio de disco (LVM)	Si activamos "High Logging", verificamos que la partición de logs no esté al límite.	<code>df -h /var/log o sudo vgs (si usamos LVM)</code>

25.2. Checklist de sospechas de intrusión y respuesta a incidentes

Este checklist se activa cuando detectamos anomalías, lentitud en el sistema o alertas de seguridad.

Acción	Explicación	Comando
Identificación de atacantes actuales	Ver qué IPs están siendo bloqueadas en tiempo real por comportamiento agresivo.	<code>sudo cscli decisions list</code> y <code>sudo fail2ban-client status sshd</code>
Análisis de intentos de acceso	Buscar intentos de fuerza bruta fallidos que el firewall aún no haya mitigado.	<code>sudo lastb -a</code>
Vigilancia de tráfico en vivo	Monitorear el rechazo de paquetes en el firewall para detectar escaneos de red.	<code>tail -f /var/log/ufw.log</code>
Aislamiento preventivo	Si detectamos una IP muy persistente, la bloqueamos manualmente en el firewall.	<code>sudo ufw insert 1 deny from <IP_ATACANTE> to any</code>
Verificación de conexiones establecidas	Revisar si hay alguna conexión activa desde ubicaciones geográficas inusuales.	<code>ss -antp</code>

25.3. Checklist de auditoría forense / post-incidente

Este checklist se ejecuta tras un incidente confirmado para entender el "cómo", limpiar el sistema y fortalecerlo.

Acción	Explicación	Comando
Auditoría de integridad profunda	Ejecutar Lynis para buscar brechas de configuración que permitieron el acceso.	<code>sudo lynis audit system --quick</code>
Búsqueda de rootkits y backdoors	Escanear archivos del sistema comparándolos con firmas conocidas.	<code>sudo rkhunter --check --sk</code>
Rastreo de persistencia (SUID)	Buscar si el atacante dejó archivos con permisos especiales para recuperar acceso root.	<code>find / -perm -4000 -type f 2>/dev/null</code>
Análisis de línea de tiempo de logs	Revisar las acciones ejecutadas por el usuario comprometido (si aplica).	<code>grep "sudo" /var/log/auth.log y analizar .bash_history</code>
Evaluación de exfiltración	Revisar el log de salida (Outgoing) del firewall para ver si hubo conexiones de tipo "baliza" hacia afuera.	<code>grep "OUT=" /var/log/ufw.log</code>
Endurecimiento post-mortem	Basado en el reporte de Lynis, aplicar los "Suggestions" prioritarios.	Revisar <code>/var/log/lynis-report.dat</code>

26. Matriz de riesgos

Esta matriz de riesgos es una actualización respecto de la matriz del Laboratorio 3. El control aplicado ahora es mucho más específico. Hemos pasado de "bloquear" a "gestionar la amenaza".

Riesgo	Impacto	Control aplicado (Lab 4)	Mejora respecto al Lab 3
Robo de credenciales	Crítico	SSH Key Authentication (ED25519)	Se eliminó el uso de contraseñas. El atacante no puede entrar aunque adivine el password.
Fuerza bruta (Bots)	Crítico	PasswordAuthentication no + CrowdSec	Los bots ya no tienen un prompt de contraseña donde atacar; son bloqueados antes de intentar nada.
Rootkits / troyanos	Crítico	RKHunter (Rootkit Hunter)	Auditoría de integridad. Detectamos si un atacante modificó archivos del sistema (binarios).
Escaneo de puertos	Medio	Fail2Ban [portscan] + [port-hammer]	El sistema detecta la fase de reconocimiento del atacante y lo banea preventivamente.
Ataque distribuido	Alto	CrowdSec (Community Blocklist)	Defensa colectiva: bloqueamos IPs maliciosas basándonos en ataques reportados por otros.
SYN Flood (DoS)	Alto	UFW before.rules (Limit 10/s)	Protección a nivel de Kernel para que el servidor no se sature por conexiones falsas.
Falsos positivos	Bajo	RKHunter Whitelist + IP Whitelist	Se ajustaron las reglas para permitir archivos legítimos de Ubuntu y el acceso del admin.

27. Arquitectura del laboratorio

Al igual que como sucede con nuestra matriz de riesgos, la arquitectura del laboratorio evoluciona y se agregan herramientas de defensa respecto del Laboratorio 3 y una herramienta de monitoreo externo como es el caso de **Crowdsec**.

Componente	Sistema / Detalle
Máquinas atacantes	Windows 11 (Host) / Kali Linux (Virtual)
Servidor protegido	Ubuntu Server 24 (LVM expandido a 30GB)
Acceso seguro	Túnel SSH con llaves ED25519 (Sin contraseñas)
Red	VMWare Bridge / Red local (IP: 192.168.0.36)
Herramientas defensa	UFW (High Logging), Fail2Ban, CrowdSec, RKHunter
Auditoría	Lynis (Hardening Index) + RKHunter Propupd
Monitoreo externo	CrowdSec Cloud Dashboard

28. Conclusiones de seguridad

El estado de hardening avanzado

Al finalizar este laboratorio, el servidor Ubuntu Server 24 ha alcanzado un estado de **Hardening Avanzado**, consolidándose como un **bastión endurecido**. Para que un auditor de seguridad considere un sistema como "Fuerte", este debe cumplir con el modelo de **Defensa en Profundidad**, y nuestro servidor ahora cumple con los cuatro niveles críticos de dicho modelo:

1. Nivel de red (perímetro y resiliencia)

- **Estado:** Mediante **UFW**, el servidor bloquea todo tráfico no autorizado y mitiga ataques de inundación (**SYN Flood**).
- **Optimización forense:** La expansión de disco mediante **LVM** y el *High Logging* permiten que, ante un eventual incidente, contemos con el historial de datos necesario para un análisis forense completo sin riesgo de desbordamiento de disco o denegación de servicio por falta de espacio.

2. Nivel de Aplicación (Identidad criptográfica)

- **Estado:** Se ha eliminado el factor humano (contraseñas). La transición a llaves **SSH (ED25519)** elimina el 99% de los ataques automatizados en internet.
- **Seguridad matemática:** Un atacante puede tener todo el tiempo del mundo, pero sin la llave privada, el acceso es matemáticamente imposible. Hemos cerrado la puerta principal a cualquier intento de fuerza bruta.

3. Nivel de Comportamiento (Defensa colaborativa e IPS)

- **Estado:** **Fail2Ban** y **CrowdSec** actúan como un sistema inmunológico activo. El servidor "siente" el ataque y reacciona cerrándose automáticamente.
- **Inteligencia colectiva:** Gracias a CrowdSec, el servidor es ahora parte de una red de inteligencia global. No necesitamos ser atacados para aprender; bloqueamos amenazas basándonos en la experiencia de la comunidad antes de que toquen nuestra red.

4. Nivel de Host (Integridad y Auditoría)

- **Estado:** Con la implementación de **RKHunter** y **Lynis**, el servidor vigila su propio "estado de salud".
- **Vigilancia interna:** Estas herramientas aseguran que, si las capas anteriores fallan, el administrador sea alertado de inmediato si algún binario crítico ha sido alterado o si algo cambió en las "entrañas" del sistema.

⚠ El factor humano: Riesgos del hardening fuerte

Un endurecimiento de este nivel traslada la responsabilidad directamente al administrador. Para mantener la operatividad, se deben gestionar dos puntos críticos:

- **Evitar la "fatiga del administrador":** Es imperativo ejecutar `sudo rkhunter --propupd` tras cada actualización legítima del sistema. De lo contrario, empezarás a recibir alertas de falsos positivos que degradarán la confianza en el monitoreo.
- **Gestión crítica de credenciales:** Al haber deshabilitado las contraseñas, la pérdida de tu llave privada significa un **lockout total**. No existe "puerta trasera"; el respaldo de tu llave es ahora tan crítico como el servidor mismo.

Nota de seguridad final:

La "Prueba de Oro" El paso más crítico de este laboratorio fue la validación del acceso SSH antes de purgar las contraseñas. Este procedimiento garantizó la continuidad del acceso administrativo y validó que la configuración criptográfica era plenamente operativa antes de eliminar el respaldo de autenticación tradicional.

